Computers & Security 173 (2027) 105225

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full length article

A newly generated dataset and pattern-driven approach for detecting AMQP Denial-of-Service attacks☆

Munmun Swain [a],*, Sushant Kuril [a], Nikhil Tripathi [b], Kamalakanta Sethi [c]

[a] Indian Institute of Information Technology Sri City, Chittoor, Andhra Pradesh, 517646, India
[b] Indian Institute of Technology (Indian School of Mines), Dhanbad, Jharkhand, 826004, India
[c] National Institute of Technology, Warangal, Telangana, 506004, India

ARTICLE INFO

Keywords:
AMQP protocol
Setup of client-server testbed
DoS attack
Intrusion detection
Anomaly detection

ABSTRACT

The Internet of Things (IoT) has been a major driver of digital transformation over the past decade. It relies on networks of smart sensor devices that communicate using lightweight protocols. Among these, the Advanced Message Queuing Protocol (AMQP) is widely adopted for its reliability and efficiency in message exchange. Due to their restricted computational and power resources, IoT devices are highly vulnerable to Denial-of-Service (DoS) attacks. Such attacks exploit weaknesses of the protocols, disrupting the communication and degrading system performance. A major challenge in securing AMQP based IoT systems is the lack of real-world data that captures different variants of DoS attacks for testing detection approaches. To address this issue, we built an IoT testbed and generated a comprehensive AMQP traffic dataset that includes normal system behavior. The dataset additionally includes traffic from different attack scenarios, such as Slow Incomplete Publish, Slow Partial Publish, Slow Queue Declare, Delayed Channel Close, and a Basic Flooding DoS attack. Furthermore, we propose a statistical anomaly detection technique using event pattern analysis, which effectively identifies normal and different variants of DoS attacks, including both Flooding and Slow Rate attacks. The experimental evaluation shows that our approach can detect attacks in real time with high accuracy, minimal computational overhead, and provides a practical solution to secure AMQP based IoT systems.

1. Introduction

The Internet of Things (IoT) is reshaping how people interact with technology. It was first introduced by Kevin Ashton in 1999 (Ashton, 1999), which enables physical objects to communicate and exchange data through wired and wireless connections. Although this extensive connectivity offers enormous potential for innovation and efficiency, it also introduces significant challenges, particularly in ensuring secure data transmission and maintaining network reliability. IoT sensor devices are designed to improve the daily lives of users and are now widely deployed in various domains, including healthcare, agriculture, smart transportation, fitness, education, industrial automation, surveillance, and military operations (Celik et al., 2019; Sharma and Saini, 2020; Kong et al., 2022; Alaba et al., 2017). These devices are typically characterized by low cost, minimal bandwidth requirements, and efficient hardware performance, enabling seamless machine-to-machine (M2M) communication.

To support the rapid growth of the IoT ecosystem, various communication models have been developed (Saini and Sharma, 2025), powered by application layer protocols such as the Constrained Application Protocol (CoAP) (Shelby et al., 2014), Message Queuing Telemetry Transport (MQTT) (Naik, 2017; Quincozes et al., 2019), and the Advanced Message Queuing Protocol (AMQP) (Vinoski, 2006). Among these, AMQP (Kramer, 2009) has emerged as one of the most reliable and widely adopted protocols, offering secure and efficient data transmission between IoT devices and brokers. AMQP introduced by John O'Hara at JPMorgan Chase in 2003 (Wiggers, 2003) has become one of the most widely adopted IoT messaging protocols owing to its reliability, standardized open design (Fernandes et al., 2013; Vinoski, 2006), and support for flexible message routing via brokers like RabbitMQ (Roy, 2017; Johansson and Dossot, 2020), ActiveMQ (Apache ActiveMQ Project, 2004), exchanges, and persistent queues in a decoupled publish–subscribe model. Apart from its standardized design for reliable messaging, the security aspects of AMQP communication remain largely underexplored in research. The highly connected nature of IoT networks poses significant security challenges (Baig et al., 2017),

☆ This research was conducted at IIIT Sri City, IIT (ISM) Dhanbad, NIT Warangal.

* Corresponding author.
E-mail addresses: munmun.s@iiits.in (M. Swain), sushant.k22@iiits.in (S. Kuril), nikhiltripathi@iitism.ac.in (N. Tripathi), kamalakanta@nitw.ac.in (K. Sethi).

https://doi.org/10.1016/j.cose.2026.105225
Received 3 January 2026; Received in revised form 10 October 2026; Accepted 25 October 2026
Available online 2 November 2026
0167-4048/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

making AMQP susceptible to advanced cyber threats such as Denial-of-Service (DoS) attacks (Chien and Wang, 2022; Tripathi, 2023; Tripathi and Shaji, 2022; Tripathi and Hubballi, 2018; Tripathi et al., 2016; Tripathi and Hubballi, 2021; Swain et al., 2025b,a; Manivannan, 2024). This threat is amplified by the increasing number of compromised IoT devices available to launch such attacks (Syed, 2020).

Although existing works have investigated the security of AMQP through formal verification, intrusion detection systems, and architectural enhancements, limited attention has been given to real-time, application layer behavioral analysis of AMQP under DoS conditions. In particular, existing approaches do not sufficiently capture dynamic protocol state transitions and event-driven interactions that characterized AMQP communication. This gap is significant because AMQP differs fundamentally from MQTT in terms of protocol complexity and operational behavior. MQTT follows a lightweight publish–subscribe model with limited interaction points, resulting in a relatively narrow application layer attack surface. In contrast, AMQP employs a multi-layered architecture involving connection establishment, channel management, queue and exchange declarations, bindings, and multi-frame message transfers. These operations create a larger state space and introduce complex protocol lifestyles that can be exploited by adversaries.

Consequently, AMQP is more vulnerable to sophisticated DoS strategies, including Slow-rate and multi-stage attacks that manipulate protocol states, such as delayed channel closures, excessive queue declarations, and fragmented message transmissions. These attack patterns cannot be effectively represented using MQTT based datasets or detection techniques. To address this limitation, this work proposes a protocol aware, event-pattern based detection framework that captures sequential behavior and state transitions in AMQP traffic. Furthermore, a standardized AMQP specific dataset is developed to model realistic DoS attack scenarios, enabling effective evaluation of application layer intrusion detection mechanisms in IoT environments.

In the first phase of this research, a controlled AMQP based IoT testbed is developed to generate a comprehensive dataset. The environment captures both normal AMQP traffic and various DoS attack traffic, including Slow Incomplete Publish, Slow Partial Publish, Slow Queue Declare, Delayed Channel Close, and Basic Flooding DoS attacks. This dataset serves as the foundation for subsequent analysis and detection.

The second phase of our research focuses on developing an AMQP attack detection approach termed *Event Pattern Analysis (EPA)*, which identifies malicious activity by analyzing network packet sequences and behavioral characteristics. Normal traffic is used to extract event patterns and generate look-ahead pair sequences, which are stored in a dedicated reference database. Likewise, the maximum observed inter-packet delay of the legitimate traffic is computed and stored in a separate delay database. During the evaluation phase, any incoming event pattern that exceeds a predefined threshold, deviates from the stored look-ahead patterns, or surpasses the normal delay range is flagged as a potential DoS attack.

The comprehensive key contributions of this work are summarized as follows:

- A dedicated IoT dataset for AMQP security is introduced, capturing both legitimate traffic and multiple variants of DoS attacks, thereby addressing a critical evaluation gap in AMQP layer intrusion detection research.
- A lightweight Event Pattern Analysis (EPA) based intrusion detection framework is proposed to identify anomalous AMQP behaviors using look-ahead event pairs and inter-event temporal delays, enabling low-overhead detection suitable for IoT environments.
- The AMQP dataset is generated using a cloud-based testbed setup to emulate realistic AMQP communication and attack scenarios. The resulting dataset is used to evaluate the practical effectiveness and deployability of the proposed approach under realistic operating conditions.

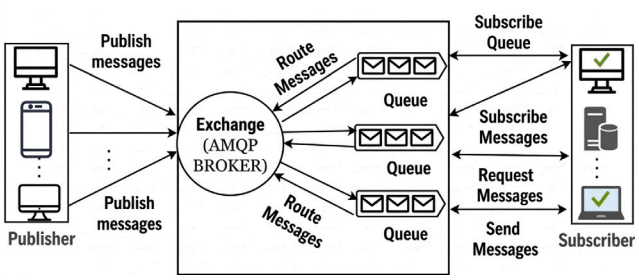


Fig. 1. AMQP publish–subscribe architecture.

- A comprehensive quantitative and qualitative comparison with existing AMQP security approaches is presented, highlighting improvements in detection accuracy, computational efficiency, and adaptability achieved by the proposed protocol-aware statistical anomaly model.
- The memory footprint and scalability of the proposed approach are evaluated, demonstrating its suitability for deployment in resource-constrained IoT environments.

The paper is organized as follows. Section 2 introduces the AMQP protocol and its role in IoT systems. Section 3 reviews existing research on IoT network attacks, highlighting AMQP specific vulnerabilities and detection methods. Section 4 details the testbed environment used for dataset generation, summaries of attack scenarios. Section 5 presents the proposed event pattern analysis approach for attack detection. Section 6 discusses the experimental results, including confusion matrices and performance analysis. Finally, Section 7 concludes the paper with key findings and future research directions.

2. AMQP background

AMQP is a standard messaging protocol that provides reliable, scalable, and secure communication for distributed systems. As introduced in Section 1, its architecture is built around a message broker, such as RabbitMQ, which manages message delivery through exchanges and queues in a publish–subscribe topology. Publishers send messages to exchanges, which forward them to bound queues based on routing keys and binding rules. Subscribers then consume and acknowledge messages asynchronously from these queues. This decoupled communication design forms the foundation for the DoS attack variants described in Section 4, while the communication-phase semantics discussed below directly support the event pattern model proposed in Section 5. Fig. 1 provides an architectural overview of AMQP.

AMQP communication begins with a client establishing a TCP connection to the broker, followed by a protocol handshake in which protocol headers, authentication credentials, and connection parameters are exchanged. After successful authentication, the client creates one or more lightweight channels that act as independent logical streams sharing the same TCP connection. These channels enable multiplexed AMQP operations and efficient asynchronous communication between publishers, exchanges, queues, and subscribers.

The broker, implemented using platforms such as RabbitMQ or Apache Qpid, manages message routing and delivery while supporting messaging patterns including publish–subscribe and point-to-point communication. Queues act as temporary buffers that store messages until they are consumed by subscribers. Exchanges and queues are configured using AMQP commands such as `Exchange.Declare`, `Queue.Declare`, and `Queue.Bind`, enabling flexible and reliable message distribution.

3. Related works

IoT security research has evolved from general traffic monitoring to protocol centric intrusion detection. Despite this progression, real-time behavioral detection of application layer DoS attacks targeting AMQP remains largely unexplored in the existing literature. Although AMQP is widely adopted within IoT environments, its susceptibility to DoS attacks has not been explored sufficiently in the existing literature. This section discusses the currently known AMQP vulnerabilities and corresponding defense mechanisms.

3.1. Existing attacks on AMQP

The existing literature indicates that AMQP introduces significant security risks, particularly within IoT messaging environments. AMQP brokers are essential for communication, yet they remain exposed to various DoS attacks. The National Vulnerability Database (NVD) (National Institute of Standards and Technology, 2025) documents these threats through various Common Vulnerabilities and Exposures (CVE) (MITRE Corporation, 2025) entries, each detailing severity and impact. However, these entries primarily report vulnerabilities and do not include real-time implementation, experimental analysis, or evaluation in a detection context. They are referenced here for completeness. While prior studies primarily describe these vulnerabilities conceptually, there is limited progress in implementing real-world attack scenarios or developing practical and effective detection mechanisms.

It is noted that the CVEs summarized below represent the single-shot implementation flaws that trigger an immediate crash. In contrast, this work targets a multi-stage, protocol-state-driven Slow-rate DoS behavior that remains within valid AMQP syntax and exploits legitimate state transitions rather than a single malformed input.

3.1.1. Malformed AMQP message injection (CVE-2021-22116 Common Vulnerabilities and Exposures (CVE), 2021b , CVE-2017-15699 Common Vulnerabilities and Exposures, 2017, CVE-2019-0200 National Institute of Standards and Technology (NIST), 2019, CVE-2015-0203 Common Vulnerabilities and Exposures (CVE), 2015)

In this attack, the attacker sends a stream of deliberately malformed AMQP frames to the broker or message router. The software fails to properly validate these anomalous packets, resulting in critical runtime errors that may crash or terminate the messaging service. As a result, legitimate clients are unable to connect, effectively causing a denial of service.

3.1.2. Out-of-memory (OOM) DoS (CVE-2021-22095 National Institute of Standards and Technology (NIST), 2021 , CVE-2020-27217 Common Vulnerabilities and Exposures (CVE), 2020, CVE-2023-46118 National Institute of Standards and Technology (NIST), 2023a)

In this attack, an attacker sends an extremely large AMQP message that exceeds the expected size limit. The vulnerable software fails to enforce this limit and attempts to fully process the message, consuming all available memory.This leads to an Out-of-Memory error that causes the broker to crash, resulting in a DoS attack.

3.1.3. Oversized message handling in Apache Qpid Broker-J (CVE-2018-8030 National Institute of Standards and Technology (NIST), 2018)

An attacker exploits this vulnerability by sending messages larger than the allowed maximum size (default 100 MB) using AMQP protocols. This causes the Apache Qpid Broker-J to crash, and form the DoS attack.

3.1.4. Memory exhaustion via large messages (CVE-2023-46120 Common Vulnerabilities and Exposures (CVE), 2023)

An attacker sends a very large message to a client using a vulnerable version of the RabbitMQ Java library. The client fails to enforce the `maxBodyLength` limit, causing it to consume excessive memory and crash with an Out-of-Memory error.

3.1.5. HTTP API message size limit bypass (CVE-2023-46118 National Institute of Standards and Technology (NIST), 2023b)

An attacker sends an extremely large HTTP request body to the RabbitMQ broker. Since the broker lacks proper size limits, it attempts to process the entire payload. This results in memory exhaustion and forces the RabbitMQ process to terminate, causing a denial-of-service condition.

3.1.6. Authentication/SASL handling flaw (CVE-2018-1298 Common Vulnerabilities and Exposures (CVE), 2018)

An attacker sends a specially crafted SASL authentication request using the PLAIN or XOAUTH2 mechanism. This triggers improper handling in the broker's authentication provider, causing the Apache Qpid Broker-J instance to crash.

3.1.7. CPU exhaustion via malicious payload (CVE-2021-22097 Common Vulnerabilities and Exposures (CVE), 2021a)

A crafted serialized object message with a specific content type is sent by the attacker to the AMQP application. When the application invokes the toString() method on this message, it enters excessive computation or an infinite loop, fully consuming CPU resources and leading to DoS attack.

3.2. Existing detection approaches

Intrusion detection research has evolved considerably over the years, progressing from traditional rule based and temporal analysis techniques to advanced machine learning, deep learning, and protocol aware security frameworks for IoT environments. Early intrusion detection systems primarily focused on identifying abnormal behavior through temporal relationships and predefined event patterns. Lee et al. (2000) proposed a time-signature based approach for real-time database systems, where anomalies were detected through deviations in task execution timing and update latency. Similarly, Dwivedi and Tripathi (2015) introduced an event correlation framework that grouped alerts according to predefined rules such as alert frequency, source and destination attributes, and alert type. Although these approaches demonstrated the importance of temporal behavior and event relationships, they were largely dependent on manually crafted rules and were not sufficiently adaptable to evolving attack patterns.

With the rapid adoption of IoT technologies, research gradually shifted toward machine learning (ML) based intrusion detection techniques capable of learning patterns directly from network traffic. Several studies explored classical ML algorithms to detect Flooding and Denial-of-Service attacks in MQTT environments. Zeghida et al. (2023) combined multiple classifiers using ensemble learning techniques such as bagging, boosting, and stacking to improve detection accuracy for Flooding attacks. Ikhsanudin et al. (2023) employed a DBSCAN-SVM hybrid framework for MQTT Flooding attack detection, while Rachmadi et al. (2021) applied AdaBoost using TCP and MQTT traffic features to identify CONNECT Flooding attacks. Siddharthan et al. (2022) further introduced the SENMQTT-SET dataset and evaluated multiple ML models using a multi-cascade feature extraction framework. Although these methods achieved promising results, they relied heavily on handcrafted traffic features and were primarily effective against volumetric Flooding attacks.

To improve efficiency and reduce feature redundancy, several studies incorporated feature selection and optimization mechanisms into ML based IDS frameworks. Imran et al. (2023) demonstrated that correlation based feature selection significantly reduced dimensionality and training time on the MQTTSET dataset. Alotaibi et al. (2024) proposed a distributed ML framework using Random Forest and XGBoost classifiers to detect DoS and brute-force attacks, while Francis et al. (2024) introduced a hybrid Genetic Algorithm Random Forest model using bi-flow traffic features. Other studies explored PCA based feature reduction (Al-Fayoumi and Al-Haija, 2023) and smart home oriented

MQTT detection pipelines (Patni et al., 2024). Despite improvements in computational efficiency and classification accuracy, these approaches continued to depend on statistical traffic characteristics and lacked the ability to capture sequential protocol behavior and dynamic event interactions.

To address the limitations of traditional ML approaches, researchers increasingly adopted deep learning (DL) techniques capable of modeling complex temporal and spatial dependencies in network traffic. Alzahrani and Aldhyani (2022) demonstrated that CNN-LSTM architectures improved intrusion detection performance by jointly learning spatial and temporal traffic patterns. Khan et al. (2021) proposed a Deep Neural Network (DNN) using uni-flow, bi-flow, and packet-flow abstractions for IoT intrusion detection, while Otoom et al. (2023) developed a DL based IDS targeting brute-force attacks using bi-flow features. Additional studies incorporated autoencoder, GAN, CNN-RNN, CNN-GRU, and Transformer based architectures to improve anomaly detection and address class imbalance issues (Boppana and Bagade, 2023; Zeghida et al., 2025; Ullah et al., 2023). Although DL approaches improved detection capability across diverse attack categories, they often introduced higher computational overhead, increased inference latency, and scalability challenges for resource constrained IoT environments.

Recent research has therefore focused on balancing detection accuracy with computational efficiency through hybrid, federated, and optimization driven frameworks. Norouzi and Gürkaş-Aydın (2026) combined Genetic Algorithm based feature selection with Ensemble Bagging Trees for intrusion detection across multiple IoT datasets. Other studies explored hybrid CNN-GRU architectures, federated learning frameworks, blockchain integrated IDS systems, and reinforcement learning based security models for Industrial IoT and Internet of Medical Things (IoMT) environments (Nandanwar and Katarya, 2024a, 2025c,d,b, 2024b, 2025a; Khan et al., 2024c,a,b; H. Kumar et al., 2025). While these approaches enhanced privacy preservation, interpretability, and distributed learning capabilities, they continued to face challenges related to scalability, communication overhead, and deployment complexity.

Beyond MQTT, researchers have also investigated security challenges in other IoT application layer protocols such as the Constrained Application Protocol (CoAP). Escamilla-Ambrosio et al. (2025) proposed a Moving Target Defense (MTD) strategy using randomized CoAP dialects to defend against spoofing and response injection attacks. Almeghlef et al. (2023) developed a decision tree based framework for detecting application layer DDoS attacks using GAN augmented datasets, while Khatua et al. (2025) proposed the DeLoC framework to identify communication delay attacks using regression and classification models. Other studies explored formal verification methods (Cai et al., 2025) and correlation aware DL architectures for UDP Flooding attacks (Hekmati et al., 2024). Collectively, these studies demonstrate that intrusion detection approaches designed for one IoT protocol cannot be directly transferred to another without accounting for protocol specific communication semantics and message exchange behavior.

Compared to MQTT and CoAP, security research specifically targeting the Advanced Message Queuing Protocol (AMQP) remains relatively less. Existing AMQP security studies primarily focus on signature based detection, formal verification, and secure communication architectures. Hashimyar et al. (2025) developed a signature based IDS using a custom AMQP testbed to emulate DDoS, brute-force, and Man in the Middle attacks. Liu et al. (2024) applied formal verification techniques using CSP and the Process Analysis Toolkit to validate AMQP message exchanges and encrypted communication correctness. S. Kumar et al. (2025) proposed a blockchain enabled AMQP communication framework integrating hybrid encryption and federated learning based anomaly detection. More recently, another study by Kuril et al. (2026) identified four classes of Slow-rate DoS attacks targeting AMQP broker resource exhaustion through TCP connection management manipulation. However, the study focused primarily on attack characterization and did not implement or evaluate any practical detection framework.

This gap, together with the CVE-based flaws discussed in Section 3.1, motivates the real-time, event-pattern based detection framework proposed in this work.

Existing IDS solutions for IoT protocols predominantly rely on statistical traffic features such as packet counts, flow volume, and byte level characteristics. While these approaches are effective for detecting high volume Flooding attacks, they are insufficient for AMQP environments where attackers frequently exploit the application layer message lifecycle, including channel creation, queue declaration, message publishing, and connection termination, while remaining within valid protocol boundaries. Consequently, stealthy low-rate attacks can evade conventional traffic based detection mechanisms by operating within acceptable transport level thresholds.

Although prior studies have improved the understanding of AMQP vulnerabilities and secure communication mechanisms, practical intrusion detection specifically tailored to AMQP protocol behavior remains underexplored. Existing approaches are often theoretical, signature dependent, or adapted from other IoT protocols without considering the dynamic multi-stage event sequences inherent to AMQP communication. Furthermore, experimental validation against realistic Slow-rate and application layer attack scenarios remains limited. These limitations highlight the need for a lightweight and protocol aware intrusion detection mechanism capable of modeling AMQP event sequences and temporal communication behavior in real-time IoT environments. The key limitations of existing approaches can be summarized as follows:

- Existing studies emphasize theoretical analysis or generic security frameworks and do not provide real-time detection capabilities for AMQP application-layer DoS attacks.
- The model based on blockchain or heavy cryptographic mechanisms require significant processing power and memory, making it unsuitable for lightweight IoT devices that rely on AMQP.
- Most research relies on limited simulations, resulting in weak evaluation of realistic AMQP DoS behaviors such as slow publish or oversized message abuse.

The above limitations demonstrate that existing AMQP security research lacks practical, real-time, and protocol-aware solutions to defend against application layer DoS attacks. To address these challenges, this work proposes a lightweight event pattern based detection framework specifically designed for AMQP. The approach provides real-time protection against application layer DoS attacks while maintaining low computational overhead, making it suitable for resource-constrained IoT environments.

4. Empirical evaluation of AMQP attacks

We conducted an experiment on a cloud-based testbed to study how a RabbitMQ broker behaves under normal conditions and various DoS attacks. This section describes the testbed setup and the roles of each virtual machine (VM), outlines the attack behaviors, and explains the analysis methods used to evaluate broker performance and analysis the impact of each attack in a controlled environment.

4.1. Testbed setup

To analyze AMQP traffic patterns and their behavior in a controlled environment, we designed a cloud based testbed consisting of four virtual machines (VMs): a publisher, an attacker, a RabbitMQ (message broker), and a subscriber. This setup allowed us to observe normal messaging operations and subsequently evaluate the impact of DoS attacks on the AMQP based enterprise messaging system. The VMs were configured with Ubuntu 22.04 LTS (64-bit) and hosted on a dual-core

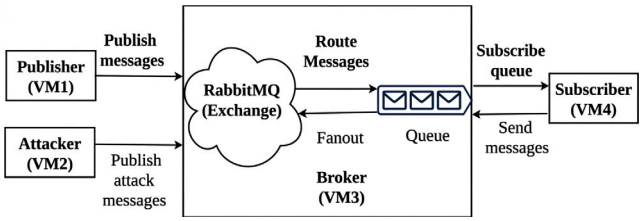


Fig. 2. AMQP testbed setup.

Intel Core i5 processor with 2 GB of allocated RAM. Fig. 2 shows the AMQP testbed setup, where the publisher is presented as (VM1), the attacker as (VM2), the RabbitMQ (message broker) as (VM3), and the subscriber as (VM4).

4.1.1. Publisher/subscriber (VM1 and VM4)

The AMQP publisher was designed to generate order messages with varying sizes and structures for the creation of the AMQP testbed dataset. The subscriber was responsible for processing the incoming messages. By employing acknowledgment mechanisms, the system ensured reliable message delivery, verified message integrity, and recorded essential processing metrics during dataset generation.

4.1.2. AMQP broker(VM3)

At the core of the testbed, the RabbitMQ broker was deployed to handle AMQP traffic on the default port 5672. Its role was to manage fundamental messaging operations, including queueing, routing, and delivery. To ensure reliability, the broker was configured with durable queues and exchanges to guaranty message persistence.

4.1.3. Attacker(VM2)

To evaluate the resilience of the AMQP protocol under adversarial conditions, a multi-threaded attack generator was deployed on a dedicated virtual machine (VM2). This custom module was designed to launch high-volume, targeted attacks by overwhelming the broker with AMQP traffic. It provided dynamic control over message size, transmission rates, and reconnection behavior, enabling realistic emulation of persistent threat actors. By generating concurrent AMQP requests, this component effectively induced significant stress on the RabbitMQ server. This helped us to evaluate its performance and stability under prolonged DoS attack conditions. The testbed supported both high volume Flooding and low-rate (Slow DoS) attacks to capture a diverse range of adversarial behaviors.

4.2. Dataset and attack variants

This section presents the methodology adopted for traffic capture, dataset creation, and the characterization of different AMQP based DoS attack variants. The Experimental traffic was captured in PCAP format using *tcpdump*, ensuring complete visibility of AMQP protocol interactions, including connection establishment, queue declarations, message exchanges, acknowledgments, heartbeat frames, and session termination. Each execution instance was labeled as either *normal* or *attack* traffic and enriched with metadata describing the attack variant, number of threads, message generation rate, payload size, and runtime duration.

Since AMQP is an application layer protocol, conventional packet inspection tools cannot directly interpret its encoded message structures. Therefore, all PCAP files were processed using a custom Python based AMQP parser designed to extract protocol level behavioral features suitable for anomaly analysis. Additionally, Wireshark was used to visualize communication patterns, packet timing behavior, and message flow characteristics during both normal and adversarial scenarios.

The generated dataset consists of normal AMQP traffic and five distinct DoS attack variants, including Basic Flooding, Slow Incomplete Publish, Slow Partial Publish, Slow Queue Declare, and Delayed Channel Close attacks. To ensure reproducibility of the dataset, Table 1 summarizes the traffic generation parameters used across both normal and attack scenarios.

Traffic was captured using `tcpdump` in PCAP format, parsed using the custom AMQP parser, and further analyzed through Wireshark visualization. Table 2 summarizes the dataset composition in terms of the number of flows, packet count, and PCAP file size for each traffic category. The captured traffic was used to construct the reference databases, comprising 6106 normal flows, along with the attack captures. For the evaluation presented in Section 6.1, an additional 10,740 legitimate flows from independent capture sessions and 7321 attack flows selected from the attack captures were used. The composition of the evaluation dataset for each attack variant is provided in Table 10.

The dataset demonstrates substantial diversity in both traffic volume and attack behavior. The high volume Flooding attacks generate a significantly larger number of flows, whereas Slow-rate attacks such as Delayed Channel Close maintain long lived persistent connections, resulting in a relatively smaller number of flows but a substantially higher packet count. These variations provide a comprehensive representation of both volumetric and stealthy resource-exhaustion attack patterns against AMQP brokers. In the following subsection, the impact of each DoS attack variant on broker behavior and communication characteristics is analyzed in detail.

4.2.1. Basic flooding attack

This attack variant establishes a large number of concurrent AMQP connections and threads that repeatedly declare exchanges and queues, and publish high-volume batches of messages to overwhelm the broker. Each thread continuously sends transactions with large payloads and short TTL values, forcing the broker to expend significant CPU, memory, and I/O resources for processing and temporary storage. The attack also involves frequent reconnection attempts, abnormal operations such as invoking `basic_get` from the publisher side, and rapid queue creation and deletion forming a distinct signature pattern. Additionally, the use of publisher confirms increases the broker's processing load. The attack module monitors local memory and maintains keep-alive heartbeats at an interval of 1200, ensuring a sustained high message rate and a large number of simultaneous connections that ultimately exhaust broker resources.

4.2.2. Slow incomplete publish attack

The attacker first performs normal TCP and AMQP handshakes to appear as a legitimate client, then issues genuine state creating commands (for example, declaring queues or exchanges and setting bindings), causing the broker to reserve internal state for those resources. The attacker advertises large message payloads but delivers data extremely slowly with long random pauses. This forces the broker to hold internal buffers and wait for transmission to complete. The keep-alive frames or periodic heartbeats are maintained to prevent connection timeouts and keep these sessions open. When multiple such sessions and repeated resource declarations occur simultaneously, the broker's connection slots, memory, and worker threads eventually become saturated, leading to resource exhaustion.

4.2.3. Slow partial publish attack

In this attack, the adversary begins by completing the TCP three-way handshake and AMQP negotiation to appear as a legitimate client, then opens multiple channels and issues valid queue or exchange declarations. The attacker advertises a large message but sends only a small initial fragment, intentionally stalling the remainder of the payload. The periodic, randomized delays and heartbeats keep the connection active, preventing timeouts. As a result, the broker must hold reserved memory and other internal resources while waiting for

Table 1
Traffic generation parameters for normal and attack scenarios.

Parameter	Normal traffic	Basic Flood	Slow Incomplete Publish	Slow Partial Publish	Slow Queue Declare
Publisher threads	1–5	50–200	10–50	10–50	10–50
Publishing rate	Configurable	Maximum rate	Slow	Slow	Slow
Message size (bytes)	128–1024	1024–65 536	65 536 (partial)	65 536 (partial)	N/A
Inter-byte delay (s)	–	–	2–5 (randomized)	–	Randomized
Heartbeat interval (s)	60	1200	1200	1200	1200
Queue/Exchange declarations	Standard	Rapid burst	Repeated	High-volume	Accumulated
Behavior	ACK with occasional reject/re-delivery	Limited	Incomplete	Partial	Not completed
Channel.Close sent	Yes	Yes	Yes	Yes	No
Capture tool	tcpdump (PCAP)	tcpdump	tcpdump	tcpdump	tcpdump
Labeling	Normal	Attack	Attack	Attack	Attack

Table 2
AMQP dataset summary.

Traffic type	#Flows	#Packets	PCAP size (MB)
Normal traffic	6106	2,784,373	770.4
Slow Partial Publish	1274	923,496	126.3
Delayed Channel close	526	6,072,057	1392.64
Slow Queue Declare	990	244,448	43.4
Slow Incomplete Publish	476	340,113	79.4
Basic Flooding DoS	30,274	2,192,161	502.6
Total	**39,646**	**12,556,648**	**2914.74**

the incomplete message. When many such partially published sessions are maintained in parallel, the broker's memory, connection slots, and worker resources become increasingly strained, ultimately leading to resource exhaustion.

4.2.4. Slow queue declare attack

In this scenario, the adversary creates a large number of long-lived AMQP connections and slowly issues queue and binding declarations at a deliberately low rate. Rather than overwhelming the broker with message volume, the attack relies on the steady accumulation of metadata and resource-intensive objects through legitimate management operations such as queue declarations and bindings. The attacker preserves connection longevity via heartbeats and sporadic activity to avoid idle timeouts and disconnects. By gradually declaring queue, potentially using excessive or inflated configuration parameters such as the broker's memory and computational resources become progressively saturated. This results in noticeable symptoms such as rising latency for queue and channel operations, a growing count of queues and channels, extended connection durations, and a reduced ability of the broker to accept new client connections

4.2.5. Delayed channel close attack

The adversary initiates a large number of long-lived AMQP connections and opens multiple channels on each. Rather than closing these channels normally, the attacker withholds the *Channel.Close* frames, leaving them in a semi-active state. Each channel performs a sequence of legitimate operations, such as declaring durable queues with long TTLs and high length limits, creating bindings, and declaring exchanges, with randomized delays to mimic normal client behavior. Over time, this accumulation of these half-open channels and associated metadata depletes broker resources, leading to performance degradation, increased latency, and eventually denial of service, even though the traffic continues to appear legitimate.

4.3. Performance evaluation of AMQP broker under DoS conditions

To evaluate the impact of DoS attacks on the broker, we monitored the following metrics.

- **Message publishing time:** The total time required to deliver a message from publisher (VM1) to broker (VM3). We logged timestamps immediately before and after invoking `basic.publish()` at 5 s intervals, with results stored in `performance_logs.csv`. Any error or exception was recorded as a failed publish.
- **Message publishing success rate:** The ratio of successful `basic.publish()` calls to total attempts. For each attempt, we recorded message size, success/failure status, and timestamps in `performance_logs.csv`, enabling analysis of system reliability under attack.

Using the above testbed setup and metrics, we studied broker behavior under different variants of DoS attacks on AMQP environment.

4.3.1. Behavior under the slow incomplete publish attack

To launch the Slow Incomplete Publish attack, the attacker transmitted a large message one byte at a time with a deliberate delay of two to five seconds between bytes. This ensured that the connection was maintained for an extended duration. To prevent timeouts, the attacker periodically transmitted heartbeat (keep-alive) frames. Additionally, the attacker performed unnecessary operations such as queue declarations, exchange creation, and binding/unbinding commands to further strain the broker's resources. Over time, multiple such incomplete connections led to memory exhaustion and resource depletion, thereby denying service to legitimate clients. Fig. 3(a) illustrates the connection rate over time during the attack execution. The attack started at approximately 80 s (red dashed line), triggering a rapid rise in the number of connections per minute. The connection rate initially stabilized around 30 connections per minute and then fluctuated between 30 and 45. Peaks exceeded 45 connections per minute at around 1200 and 1600 s. At approximately 1900 s (green dashed line), the attacker stopped the attack, after which the connection rate sharply dropped to 0.

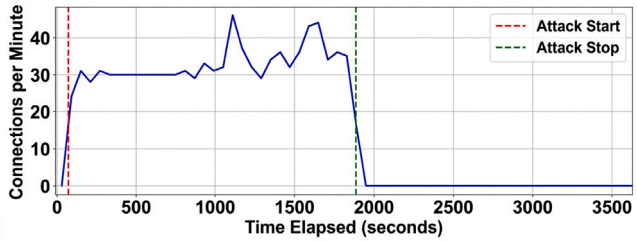
Fig. 3(b) shows that the publishing time varied between 50 and 60 ms under normal operation. Once the attack began, delays fluctuated between 50 and 90 ms for about 550 s, after which publishing time became effectively infinite (messages were no longer delivered). Fig. 3(c) shows that the publishing success rate dropped to zero at the same point, and the broker became completely unavailable. Even after the attack stopped at 1900 s, the broker did not recover and required a service restart.

4.3.2. Behavior under the slow partial publish attack

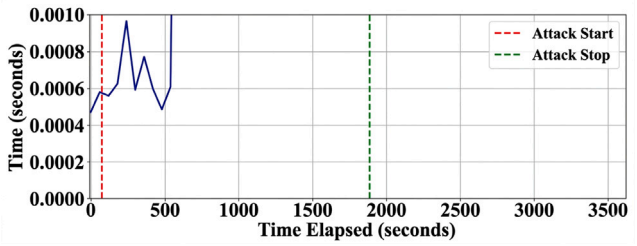
In this variant, the attacker established multiple channels, declared queues and exchanges with unusual configurations, and initiated `basic.publish()` operations by sending method and header frames while transmitting only a small fragment of the message body. This forced the broker to allocate buffers and maintain state indefinitely. Fig. 4(a) shows the impact. Publishing time became effectively infinite after about 2700 s (Fig. 4(b)), while the success rate dropped to zero at the same point (Fig. 4(c)).

4.3.3. Behavior under the slow queue declare attack

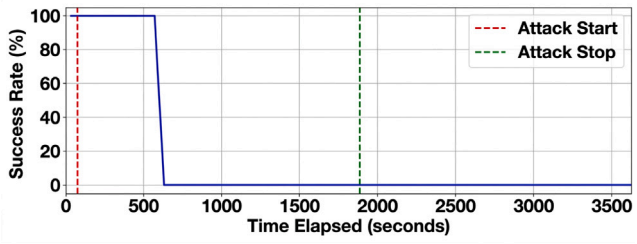
Here, the attacker exploited queue management by declaring numerous queues with memory-intensive parameters. Fig. 5(a) shows that once the attack started (80 s), the connection rate increased sharply and fluctuated between 28 and 32 connections per minute. At 450 s,



(a) Connection rate over time



(b) Message publishing time



(c) Message publishing success rate

Fig. 3. AMQP behavior under the slow incomplete publish attack. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

publishing time became effectively infinite (Fig. 5(b)), and the success rate dropped to zero (Fig. 5(c)). Neither recovered even after the attack ended at 900 s.

4.3.4. Behavior under the delayed channel close attack

In this case, the attacker opened numerous channels but never sent `Channel.Close`, leaving them semi-active while performing occasional legitimate operations. Fig. 6(a) shows that connections quickly increased to between 50 and 62 per minute, overwhelming capacity. At 350 s, publishing time became infinite (Fig. 6(b)) and the success rate dropped to zero (Fig. 6(c)), with no recovery after the attack ended at 800 s.

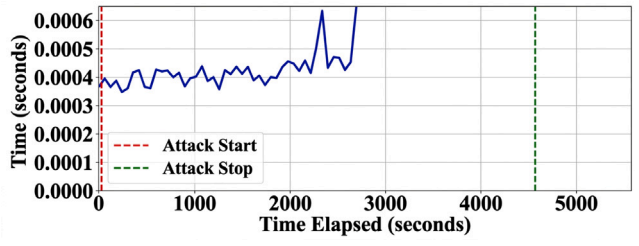
4.3.5. Behavior under the basic flood attack

In this attack, the adversary rapidly created large bursts of short-lived connections and message-publishing attempts, overwhelming the broker far more aggressively than in the preceding slow attacks. In Fig. 7(a), once the attack began at around 90 s (red dashed line), the connection rate surged to nearly 580 connections per minute by about 150 s, pushing the broker into saturation. This aggressive load caused message publishing time (Fig. 7(b)) to rise sharply toward 1 ms, and the success rate (Fig. 7(c)) to collapse from 100% to 0% by roughly 200 s. Even after the attack stopped at approximately 230 s (green dashed line), none of the metrics recovered: publishing time remained high, success rate stayed at 0%, and the system failed to return to normal operation. This demonstrates that the flood induced a persistent failure state.

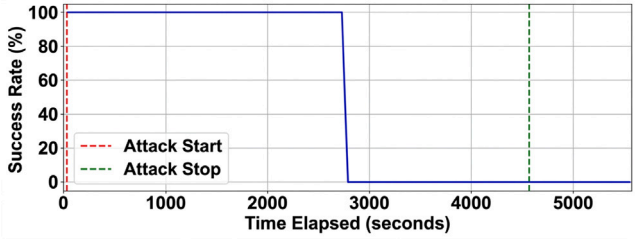
Overall resource utilization and comparison: We monitored CPU, memory, and network I/O with a `psutil` based (Rodola, 2009)



(a) Connection rate over time

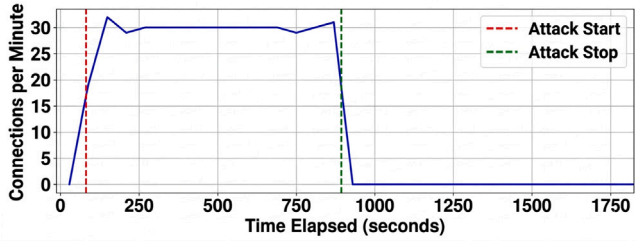


(b) Message publishing time

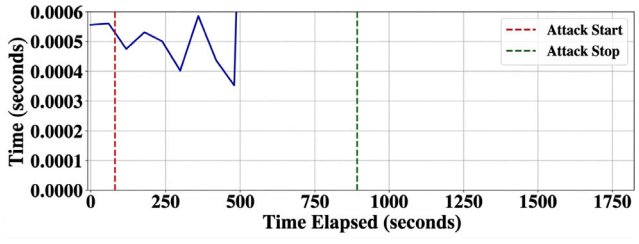


(c) Message publishing success rate

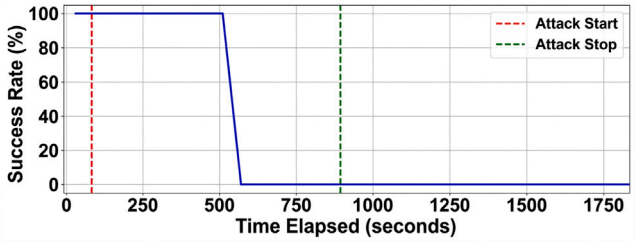
Fig. 4. AMQP behavior under the slow partial publish attack.



(a) Connection rate over time



(b) Message publishing time

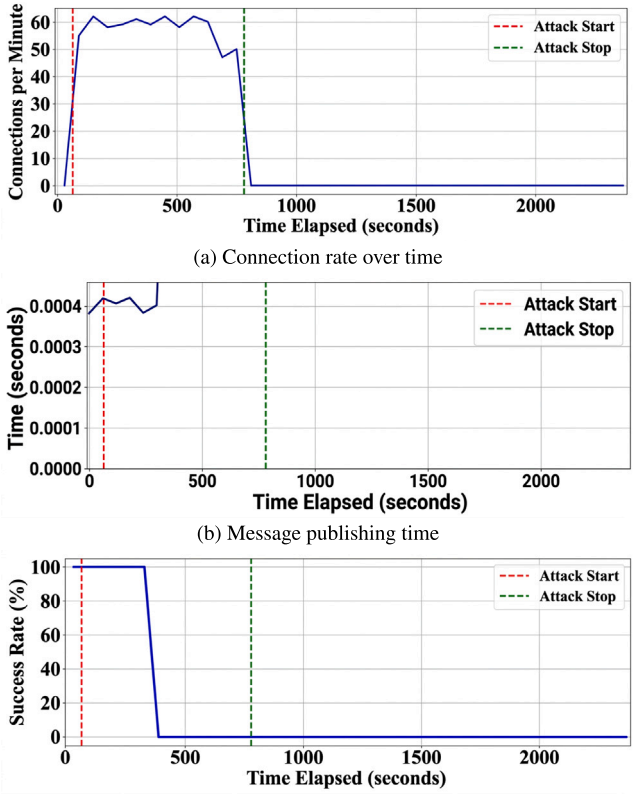


(c) Message publishing success rate

Fig. 5. AMQP behavior under the slow queue declare attack.

Table 3
Resource consumption analysis of different Slow Rate DoS attack variants on AMQP broker.

Operation state	CPU utilization (%)	Memory utilization (%)	Bandwidth sent (MB/s)	Bandwidth received (MB/s)	Critical timelines & Behavioral Patterns
Normal Operation	Stable baseline (5%–8%)	Consistent allocation (25%)	Minimal flow (0.1)	Steady low volume (0.1)	N/A
Slow Incomplete Publish	Frequent spikes (peak: 40%)	Steady increase: 69% → 71%	Stable baseline; burst at 1200 s	Sustained high volume (+20% post-700 s)	800 s: Initial degradation; 1200 s: Peak resource strain
Slow Partial Publish	Recurrent bursts (60%–90%) 5–7 spikes/min	Periodic saturation to 100%	Consistently low (<0.5)	Minor fluctuations (±0.2)	Activity concentrated pre-600 s; Complete cessation post-600 s
Slow Queue Declare	Low baseline; peak (70%) at 1450 s	Gradual growth: 25% → 55% → plateau	Minimal; terminal spike	Sustained high volume (peak: 6 MB/s) until 1300 s	250 s: Traffic surge; 300–1300 s: Peak load
Delayed Channel Close	Gradual rise to 40%; spikes during 200–400 s	Immediate saturation to 100% in 100 s	Low stable (<0.3)	Extreme burst (25 MB/s at 300 s) → flatline	50–400 s: Active attack; Memory exhaustion at 150 s
Basic Flooding Attack	Rapid surge (peaks: 75%–95%)	Fast saturation (70% → 100%)	High burst traffic during 150–200 s	Sharp increase due to excessive connection attempts	150–200 s: Success rate collapse; 180 s: Peak basic flooding rate (580 CPM); 250 s: Attack stops but broker remains degraded

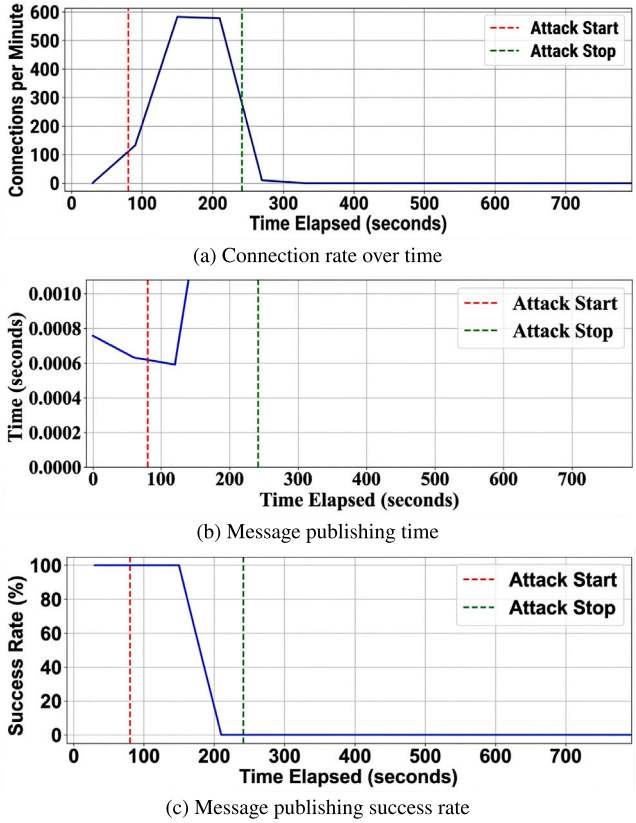


(a) Connection rate over time

(b) Message publishing time

(c) Message publishing success rate

Fig. 6. AMQP behavior under the delayed channel close attack.



(a) Connection rate over time

(b) Message publishing time

(c) Message publishing success rate

Fig. 7. AMQP behavior under the basic flood attack. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

logger. Results (Table 3) show that under attack, CPU and memory utilization spiked significantly compared to baseline. Bandwidth received also rose sharply (due to partially open connections), while bandwidth sent remained stable. These results demonstrate how Slow Rate DoS gradually overwhelmed broker resources, degrading performance without an immediate crash.

5. Proposed detection approach

In this section, we present our proposed detection approach using event pattern analysis. Fig. 8 provides an abstract overview of this proposed approach by illustrating how AMQP event patterns are captured, transformed into look-ahead pairs, compared against the normal database, and finally evaluated through the mismatch-score for anomaly classification. The methodological innovation of EPA lies

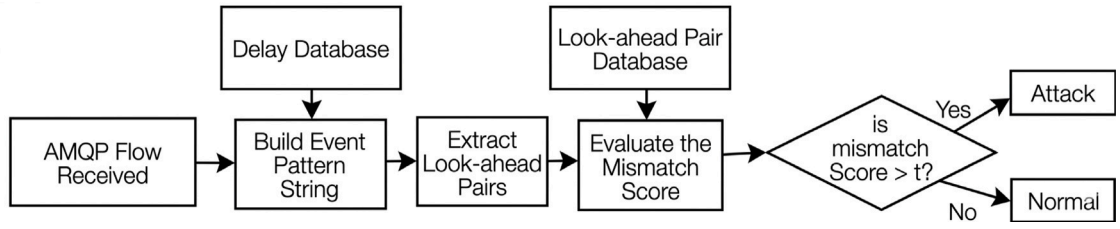


Fig. 8. Abstract overview of proposed approach.

in extending the event pattern method to AMQP significantly more complex, multi-stage protocol workflow capturing channel creation, exchange/queue declarations, and multi frame publish behavior. By integrating AMQP specific look-ahead frame sequences with a delay tracking database, the model detects syntactically valid yet timing anomalous attacks unique to AMQP. This demonstrates, for the first time, that a lightweight, statistical detector can operate effectively within AMQP richer state space.

When an AMQP request is initiated, the broker and client engage in a sequence of events that follow specific patterns. These patterns vary depending on factors such as the event type, the number of event exchanges, and the broker's response. Despite this diversity, only unique AMQP event patterns are stored in a database representing normal protocol behavior, serving as a reference to identify potential threats. If a new event pattern does not match any existing database entry, it may be flagged as suspicious. However, this approach can lead to false positives, as some rare but legitimate behaviors might be misclassified as malicious. As noted in Section 6.2.1, the wide range of normal AMQP interactions makes it impractical to compile a complete database of all legitimate behaviors. Furthermore, storing only unique events is insufficient for distinguishing between normal and attack variants. To address this limitation, incorporating the time intervals between successive events could provide an additional feature to improve the accuracy of anomaly detection.

So, we maintain a look-ahead pairs database to store normal AMQP event patterns (Chandola et al., 2010; Tripathi, 2023). These pairs aid in detecting malicious activity by highlighting deviations in event patterns. As explained in Section 6.2.1, pairs are generated using various window sizes to better represent the characteristics of normal AMQP interactions. During the evaluation phase, each new event pattern is assigned a mismatch-score.

If any look-ahead pairs in the pattern are not found in the normal database and the mismatch-score exceeds a predefined threshold, the system marks the pattern as malicious.

5.1. Pre-processing phase

During the pre-processing phase, we design two separate normal databases. The first database, denoted as $\mathcal{D}_{Look-ahead}$, stores the look-ahead pairs that are extracted from normal AMQP event patterns. The second database, referred to as $\mathcal{D}_{delay}$ keeps track of the maximum possible delay observed between any two consecutive AMQP events under normal conditions. The complete steps involved in generating these two databases during pre-processing are presented in Algorithm 1. The input and output variables of Algorithm 1 are discussed in Table 4 in a detailed manner.

Table 4
Input and output variables.

Input	Explanation
$\mathcal{F}$	AMQP flows
n	Window frame size
$\mathcal{D}_{\text{Look-ahead}}$	Look-ahead pairs database
$\mathcal{D}_{delay}$	Maximum normal delay
t	Threshold for mismatch-score
f	Incoming TCP flow

Algorithm 1 Pre-processing Phase

Input: $\mathcal{F}$, n
Output: $\mathcal{D}_{Look-ahead}$, $\mathcal{D}_{delay}$

$\mathcal{D}_{Look-ahead}$={}; $\mathcal{D}_{delay}$={};
for $flow$ in $\mathcal{F}$ **do**
  $pattern$='$Start \rightarrow *$';
  for $event$ in $flow$ **do**
    $pattern$+='$\rightarrow event \rightarrow *$';
  end for
  $pattern$+='$\rightarrow Stop$';
  l=$extractLook-aheadPairs(pattern, n)$
  $\mathcal{D}_{Look-ahead}.append(l)$
  $events = extractevents(pattern)$;
  m=1;
  while $m <= length(event) - 1$ **do**
    $delay$=$events[m].time()$-$events[m-1].time()$;
    p='$events[m\text{-}1] \rightarrow events[m]$';
    if p not in $\mathcal{D}_{delay}$ **then** $\mathcal{D}_{delay}$[p]=$delay$;
    else
      $\mathcal{D}_{delay}$[p]=$max(delay, \mathcal{D}_{delay}$[p]);
    end if
    m++;
  end while
end for
return ($\mathcal{D}_{Look-ahead}, \mathcal{D}_{delay}$);

5.2. Formal state transition model of AMQP communication

Before describing how the look-ahead pair and delay databases are extracted from normal traffic, we state which event orderings AMQP permits. We model legitimate AMQP communication as a deterministic finite automaton with a partial transition function,

$$\mathcal{A} = (S, \Sigma, \delta, s_0, F), \tag{1}$$

where:

- $S = \{S_{\text{conn}}, S_{\text{chan}}, S_{\text{exch/queue}}, S_{\text{publish}}, S_{\text{close}}\}$ is the set of protocol-phase states, corresponding respectively to connection establishment, channel management, exchange/queue declaration, message transfer, and termination;
- Σ is the finite alphabet of the 25 AMQP events listed in Table 6 (*Start*, *Protocol_Header*, *Connection.Start*, ..., *Stop*). The separator $*$ and the timeout tokens TO_m are not members of Σ; TO_m is introduced only at detection time (Algorithm 2);
- $\delta : S \times \Sigma \rightarrow S$ is a partial transition function encoding the event orderings of the AMQP specification as observed in the normal flows; an event with no defined transition in a state is not permitted in that state;
- $s_0 = S_{\text{conn}}$ is the initial state, entered on the *Start* event (TCP three-way handshake);
- $F = \{S_{\text{close}}\}$ is the accepting state, in which the *Stop* event (TCP connection close) is consumed.

Table 5
Transition function δ of the AMQP automaton $\mathcal{A}$.

Current state	Triggering event	Next state
S_{conn}	Start	S_{conn}
S_{conn}	Protocol Header; Connection.Start; Connection.Start OK; Connection.Tune; Connection.Tune OK; Connection.Open	S_{conn}
S_{conn}	Connection.Open OK	S_{chan}
S_{chan}	Channel.Open	S_{chan}
S_{chan}	Channel.Open OK	$S_{\text{exch/queue}}$
$S_{\text{exch/queue}}$	Exchange.Declare; Exchange.Declare OK; Queue.Declare; Queue.Declare OK; Queue.Bind; Queue.Bind OK	$S_{\text{exch/queue}}$
$S_{\text{exch/queue}}$	Basic.Publish	S_{publish}
S_{publish}	Basic.Publish; Basic.Deliver; Basic.Ack; Basic.Reject	S_{publish}
S_{publish}	Exchange.Declare; Queue.Declare; Queue.Bind	$S_{\text{exch/queue}}$
$S_{\text{exch/queue}}$; S_{publish}	Channel.Close	S_{close}
S_{close}	Channel.Close OK; Connection.Close; Connection.Close OK	S_{close}
S_{close}	Stop	S_{close} (accepting)

Table 5 enumerates δ as the set of (current state, triggering event, next state) rules. Two features of δ correspond to the attack behavior analyzed in Section 4.2. (i) δ permits a self-transition on $S_{\text{exch/queue}}$ for repeated *Queue.Declare/Exchange.Declare* events, the legal pathway abused by the Slow Queue Declare attack through excessive repetition; and (ii) δ permits a return transition from S_{publish} to $S_{\text{exch/queue}}$ via further declarations, one of the legal pathways abused by the Basic Flooding attack through rapid repetition.

$\mathcal{A}$ is a descriptive reference model: detection itself is performed by the databases $\mathcal{D}_{Look-ahead}$ and $\mathcal{D}_{delay}$ (Algorithm 2), and δ is not executed at run time. δ is derived from the AMQP specification and the normal flows; we do not claim it covers every legitimate behavior (the number of unique normal patterns keeps growing with the number of flows, Fig. 9(a)). The event patterns of the normal flows used in pre-processing are intended to correspond to accepting runs of $\mathcal{A}$. The relation to the two databases is as follows. A look-ahead pair $(event[a], event[b], k)$ can be legal only if $event[b]$ is reachable from $event[a]$ through transitions of δ, and it is accepted by the detector only if it is also present in $\mathcal{D}_{Look-ahead}$. An inter-event delay is treated as anomalous when it exceeds the maximum recorded in $\mathcal{D}_{delay}$ for the corresponding consecutive event pair, which acts as the empirical weight of the matching transition of $\mathcal{A}$. Because Slow Queue Declare and Basic Flooding use transitions that δ permits, they are detected through the look-ahead and delay databases rather than through violations of protocol syntax. The EPA framework is thus a bounded-window, delay-annotated empirical approximation of $\mathcal{A}$, learned from traffic rather than specified a prior.

5.2.1. Generating the look-ahead pairs database ($\mathcal{D}_{Look-ahead}$)

During the pre-processing phase, an AMQP flow $\mathcal{F}$ is constructed by aggregating the payloads obtained from AMQP packets transmitted over a single TCP connection. This phase requires two inputs: the predefined window size n and the set of all captured AMQP flows. We have initialized two empty databases: $\mathcal{D}_{Look-ahead}$ to store look-ahead pairs for each flow, and $\mathcal{D}_{delay}$ to store delay values for consecutive event pairs in **(Step 1)**. Each individual AMQP $flow$ is processed individually to generate a corresponding event pattern **(Step 2)**. The pattern begins with $'Start \rightarrow *'$, to represent the initial TCP three-way handshake **(Step 3)**. Subsequently, Deep Packet Inspection (DPI) is employed to identify and extract AMQP events from the given $flow$ **(Step 4)**. These extracted events are then mapped to their respective string representations (refer to Table 6) and appended to the *pattern* **(Step 5)**. Once all AMQP events are processed, the symbol $' \rightarrow Stop'$ is appended to the *pattern* to indicate the termination of the TCP connection **(Step 7)**.

Consider an event pattern sample as follows:

$$e1 \rightarrow * \rightarrow e2 \rightarrow e3 \rightarrow * \rightarrow e4 \rightarrow * \rightarrow e5$$

Table 6
AMQP event names and their explanations.

Explanation	Event name
TCP handshake initiation	*Start*
AMQP protocol header sent	*Protocol Header*
Connection start	*Connection.Start*
Connection start acknowledgment	*Connection.Start OK*
Connection tune	*Connection.Tune*
Connection tune acknowledgment	*Connection.Tune OK*
Open connection	*Connection.Open*
Open acknowledgment	*Connection.Open OK*
Open channel	*Channel.Open*
Channel open acknowledgment	*Channel.Open OK*
Declare exchange	*Exchange.Declare*
Exchange declare acknowledgment	*Exchange.Declare OK*
Declare queue	*Queue.Declare*
Queue declare acknowledgment	*Queue.Declare OK*
Bind queue	*Queue.Bind*
Bind acknowledgment	*Queue.Bind OK*
Publish message	*Basic.Publish*
Deliver message	*Basic.Deliver*
Acknowledge message	*Basic.Ack*
Reject message	*Basic.Reject*
Close channel	*Channel.Close*
Channel close acknowledgment	*Channel.Close OK*
Close connection	*Connection.Close*
Connection close acknowledgment	*Connection.Close OK*
TCP connection close	*Stop*

In the given AMQP events pattern, different event types are separated by the symbol $*$, and the symbol $\rightarrow$ denotes the transition from one event to another of the same event type.

Once the normal event patterns are generated, we extract the look-ahead pairs, which are then stored in $\mathcal{D}_{Look-ahead}$ **(Steps 8 and 9)**. To perform this extraction, we slide a window of size $n+1$ across the event pattern. Within each window, we observe the sequence in which the events occur. For this example, we consider a window size of $n = 3$, and apply it to the previously discussed event pattern. As the window slides across the pattern, for each event within it, we log the subsequent events occurring at positions 1,2 up to n. For instance, in the first window n_1, position 1 (*) to position 3 (*e3*) in the pattern, we derive the look-ahead pairs that are shown in Table 7.

By sliding the window across the entire event pattern, we create a look-ahead pair database, denoted as ($\mathcal{D}_{\text{Look-ahead}}$), as shown in Table 8.

The look-ahead pairs are recorded in the database using the format '$event[a] : event[b], k$', where k represents the positional distance between the ath and bth events within an event pattern and is constrained to a maximum value defined by the window size n. Once the database

Table 7
Look-ahead pairs for n_1.

	Position-1	Position-2	Position-3
$e1$	$*$	$e2$	$e3$
$*$	$e2$	$e3$	
$e2$	$e3$		

Table 8
Look-ahead pairs obtained from the whole event pattern.

	Position-1	Position-2	Position-3
$e1$	$*$	$e2$	$e3$
$*$	$e2$, $e4$, $e5$	$e3$, $*$	$*$, $e5$
$e2$	$e3$	$*$	$e4$
$e3$	$*$	$e4$	$*$
$e4$	$*$	$e5$	

$\mathcal{D}_{Look-ahead}$ is created, it serves as a reference for the detection mechanism. During the detection phase, this database is used to calculate a mismatch-score for any new upcoming event pattern and to identify deviations from the established norm.

5.2.2. Creating the maximum delay database ($\mathcal{D}_{delay}$)

In the second phase of pre-processing, we construct another database, named $\mathcal{D}_{delay}$, which records the maximum normal delays observed between two consecutive events (denoted as ath and bth events). This database plays a crucial role in detecting malicious behavior where an attacker may reproduce legitimate AMQP *event pattern* but introduces abnormal delays between events. To build $\mathcal{D}_{delay}$, we first extract individual events from the sequence and store them in a list called *events* **(Step 10)**. We then iterate through this list, and for each consecutive event pair $(m-1)$th and mth, we compute the delay between their occurrences **(Step 13)**. Each computed $delay$ is stored in $\mathcal{D}_{delay}$ as a $key : value$ pair, where the key represents the transition between two events in the form '$events[m-1] \rightarrow events[m]$', and the value corresponds to the measured delay **(Step 14)**. If a key does not already exist in $\mathcal{D}_{delay}$, it is added **(Step 15)**. If it does exist, we compare the new delay with the existing value and retain the maximum of the two **(Step 17)**. This ensures that $\mathcal{D}_{delay}$ always contains the maximum observed normal $delay$ for every pair of consecutive events.

5.3. Evaluation phase

Algorithm 2 presents the actions taken to identify the normal/attack event patterns during the evaluation phase. To process and analyze the incoming flows, our method employs three specialized threads, each dedicated to a distinct component of the detection pipeline.

Thread 1 captures the packet & event extraction. The incoming TCP flow → DPI based AMQP frame parsing → map to event string (Table 6) → Append event to pattern sequence. while the second thread tracks inter event delays, checking each new event appended → Query $\mathcal{D}_{\text{delay}}$ for (last_event, max_delay) pair → If elapsed time > max_delay, inject a TO_m timeout token into the pattern → Repeat until the next event arrives.

The third thread evaluates and updates the mismatch-score for the evolving event pattern each time a new event is appended, determining whether the flow exhibits normal or attack behavior. On each pattern update → If |pattern| > n, extract look-ahead pairs → For each pair not found in $\mathcal{D}_{\text{Look-ahead}}$, increment mismatch count → Compute

$$\text{mismatch-score} = \frac{\text{mismatches}}{n\left(N - \frac{(n+1)}{2}\right)}.$$

If mismatch-score $\geq t$, flag as *Attack*; if mismatch-score $< t$ and the last event is `Stop`, flag as *Normal*

Algorithm 2 Evaluation Phase

Input: $\mathcal{D}_{Look-ahead}$, $\mathcal{D}_{delay}$, n, t, f
Output: *pattern* is normal/attack

1: $pattern$ = '';
2: **while** True **do**
3: $event$ = incoming event in f;
4: $event$+='→ $event$ →∗';
5: $events.append(event)$;
6: $last_event$=$events[-1]$;
7: $max_delay = max(\mathcal{D}_{delay}[\}last_event \rightarrow \mathcal{E}'])$ where $\mathcal{E}$ refers to all possible events after $last_event$
8: **if** $(current.time()\text{-}last_event.time())$ > max_delay **then** $pattern$+='→ TO_m →∗' where $m = 1, 2, ...$;
9: **end if**
10: $mismatch-score$=0;
11: **if** $length(events) > w$ **then**
12: p=$event.extractLook-aheadPairs()$;
13: **for** $pair$ in p **do**
14: **if** $pair$ not in $\mathcal{D}_{Look-ahead}$ **then** $mismatch-score$+=1;
15: **end if**
16: **end for**
17: $mismatch-score = mismatch-score/\ (n * (len(events) - (n+1)/2))$;
18: **if** $mismatch-score \geq t$ **then** $pattern$ is malicious;
19: **else if** $mismatch-score < t$ **and** $events[-1] == \}Stop'$ **then**
20: $pattern$ is normal;
21: **end if**
22: **end if**
23: **end while**

This score is recalculated each time a new event is appended to the pattern. For basic analysis, we consider a single TCP flow, denoted as f, for which an event pattern is generated and subsequently classified as either normal or malicious. The required inputs for our Algorithm 2, includes the look-ahead pair database ($\mathcal{D}_{Look-ahead}$), the delay distance database ($\mathcal{D}_{delay}$), a predefined mismatch-score threshold (t), the window size (n), and the incoming TCP flow (f), as detailed in Table 4.

In the evaluation phase, the proposed method continuously monitors and collects packets from an ongoing flow f, extracting AMQP events from the captured packets to incrementally construct an event pattern **(Steps 3–4)**. The system actively tracks the most recently appended event, referred to as ($last_event$) **(Steps 5–6)**. Similarly, the delay threshold database $\mathcal{D}_{delay}$ is consulted to determine the maximum acceptable delay (max_delay) between $last_event$ and any possible subsequent AMQP event pattern **(Step 7)**. If no new event is added to the pattern within this max_delay time series, a randomly generated timeout string is inserted into the pattern **(Step 8)**. This process continues, repeatedly appending timeout strings until a new AMQP event is received. The motivation behind this mechanism is to rapidly increase the mismatch-score for *pattern* associated with Slow Rate DoS attack flows, enabling their early detection.

Once the *pattern* length exceeds the specified window size n, look-ahead pairs are derived from it **(Step 12)**. Each of these extracted pairs is checked for presence in the pre-constructed look-ahead pair database $\mathcal{D}_{Look-ahead}$. If a pair is not found in the database, the mismatch count for the pattern is incremented by one **(Step 14)**. After all look-ahead pairs have been evaluated, a final mismatch-score is assigned to the *pattern* **(Step 17)**, as further explained in (Section 5.3.2). If this mismatch-score exceeds the threshold t, the *pattern* is flagged as an attack **(Step 18)**. On the other hand, if the connection terminates and the mismatch-score stays below t, the *pattern* is deemed normal **(Steps 19–20)**.

Table 9
Look-ahead pairs obtained from the test pattern.

	Position-1	Position-2	Position-3
$e1$	$*$	$e2$	$*$
$*$	$e2$, e3, $e4$, $e5$	$*$	e3, $e4$, $e5$
$e2$	$*$	$e3$	$*$
$e3$	$*$	$e4$	$*$
$e4$	$*$	$e5$	

5.3.1. Security analysis: Evasion constraints

For a protocol aware adversary to evade the EPA framework through delay modulation, three constraints must be simultaneously satisfied.

First, every look-ahead pair generated from the attack event sequence must exist in $D_{Look-ahead}$. This requires the attacker to faithfully replicate the complete legitimate AMQP interaction sequence including `Channel.Close`, normal `Queue.Declare` counts, and proper binding operations without introducing any out-of-order or anomalous event pairs. This directly prohibits the core operations that define our five attack variants, such as omitting `Channel.Close` (Delayed Channel Close attack) or issuing excessive `Queue.Declare` commands (Slow Queue Declare attack).

Second, all inter-event delays must remain strictly below the corresponding maximum values recorded in D_{delay}. Since the fundamental mechanism of Slow-rate attacks is the introduction of abnormally long inter-event delays, for example, 2–5 s per byte in the Slow Incomplete Publish attack compared to normal delays well below 1 s an attacker who complies with this constraint eliminates the very stalling mechanism that causes broker resource exhaustion.

Third, to achieve meaningful service degradation, the attacker must maintain a large number of simultaneously open resource holding connections. However, reducing inter-event delays to comply with the second constraint shortens individual session durations toward normal values, which reduces the number of concurrently open resource-holding sessions below the threshold required for measurable broker exhaustion.

These three constraints are mutually incompatible for effective Slow-rate DoS execution: an attacker who fully satisfies all three simultaneously cannot achieve meaningful service disruption. In other words, a perfectly evasive Slow-rate attack is an ineffective one, and an effective Slow-rate attack remains detectable by EPA.

5.3.2. Assigning the mismatch-scores

The database $D_{Look-ahead}$ contains entries presented in Table 8, and the following event pattern is generated from the AMQP request in the evaluation phase:

$$e1 \rightarrow * \rightarrow e2 \rightarrow * \rightarrow e3 \rightarrow * \rightarrow e4 \rightarrow * \rightarrow e5$$

By considering the window size $n = 3$, Table 9 shows the look-ahead pairs generated from the event pattern.

The look-ahead pairs highlighted in red in Table 9 represent those that do not appear in Table 8. These absence pairs are identified as mismatches. To quantify the mismatch level of an event pattern, we calculate the ratio of the number of mismatched pairs to the total number of possible mismatches. In particular, for an event pattern of length $\mathcal{N}$ within a sliding window of size n, the maximum number of mismatches can be determined using the formula provided in Forrest et al. (1996).

$$n(\mathcal{N} - n) + (n-1) + (n-2) + \cdots + 1 = n(\mathcal{N} - (n+1)/2)$$

In the given example, the mismatch-score of the event pattern is found to be $7/21 (= 0.33)$. This value is recalculated each time a new event is added to the ongoing pattern. The evaluation continues when one of the following two conditions is met:

- If the mismatch-score exceeds a defined threshold, the pattern is flagged as malicious.
- If the special event $Stop$ is appended to the pattern. In this case, if the mismatch-score remains below the defined threshold, the sequence is treated as normal. Otherwise, it is classified as malicious.

5.3.3. Timeout insertion and mismatch-score calculation

To further illustrate the algorithmic mechanisms for detecting the DoS attacks mainly, timeout token insertion (Algorithm 2, Step 8) and the resulting mismatch-score escalation, we present a concrete working example based on a Slow Partial Publish attack scenario.

Scenario: Slow Partial Publish Attack

Assume the detection system is monitoring an incoming TCP flow and has observed the following partial AMQP event sequence up to time $t = 10$ s:

Start →* → Protocol_Header →* → Connection.Start →* → Connection.Start_OK →* → Connection.Tune →* → Connection.Tune_OK →* → Connection.Open →* → Connection.Open_OK →* → Channel.Open →* → Channel.Open_OK →* → Queue.Declare →* → Queue.Declare_OK →* → Basic.Publish

This sequence reflects the normal AMQP two-phase handshakes followed by the opening of a channel, a queue declaration, and the advertisement of a large message payload. At this point, the attacker intentionally stalls only the method and header frames of `Basic.Publish` have been transmitted, and no body bytes follow.

Timeout Detection. The system consults D_{delay} and determines that the maximum normal inter-event delay following a `Basic.Publish` event is approximately 0.8 s. Since no new AMQP event arrives within this interval, Algorithm 2 (Step 8) inserts a synthetic timeout token TO_1 into the evolving event pattern:

$$\ldots \rightarrow^* \rightarrow \texttt{Basic.Publish} \rightarrow^* \rightarrow \text{TO}_1 \rightarrow^* \rightarrow \text{TO}_2 \rightarrow^* \rightarrow \text{TO}_3 \rightarrow \ldots$$

The timeout tokens are inserted at every subsequent max_delay interval for as long as no new AMQP event is received from the flow. This mechanism converts an observable temporal anomaly, an abnormally long inter-event silence into a structural anomaly within the event pattern, making it detectable via the look-ahead pair comparison.

Why Timeout Tokens Always Cause Mismatches? Timeout tokens (TO_m) never appear in $D_{\text{Look-ahead}}$, which is constructed exclusively from normal AMQP flows in which all transactions complete promptly. Consequently, every look-ahead pair that contains a timeout token whether at position-1, position-2, or position-3 within the sliding window is guaranteed to be absent from $D_{\text{Look-ahead}}$ and is therefore counted as a mismatch.

Mismatch-Score Computation

With window size $n = 3$, the pattern contains 13 legitimate events and three inserted timeout tokens, i.e. 16 events. As in Section 5.3.2, N counts all pattern elements including the $*$ separators, so $N = 16+15 = 31$ and the maximum number of look-ahead pairs is

$$n\left(N - \tfrac{n+1}{2}\right) = 3 \times (31 - 2) = 87.$$

Every pair that contains a timeout token, whether as the earlier or the later element, is absent from $D_{Look-ahead}$. Counting the pairs within distance three of TO_1, TO_2 and TO_3 gives 12 such mismatches, so

$$\text{mismatch-score} = \frac{12}{87} \approx 0.14.$$

With threshold $t = 0.03$ (Table 10, Interval 1), $0.14 \gg t$, so the flow is flagged as a Slow-rate DoS attack before the TCP connection terminates. A legitimate flow with no timeout tokens yields a score in $[0, 0.02]$, consistent with Fig. 10.

5.4. Performance complexity

Notation. Let N be the number of symbols (AMQP events, separators $*$, and inserted timeout tokens TO_m) in the event pattern of a flow, n the window size, F the number of concurrently active flows, and λ the aggregate arrival rate of symbols over all active flows. Let Σ_D be the symbols that can appear in the reference databases, namely the 25 AMQP events of Table 6 and the separator $*$, so $|\Sigma_D| = 26$. Timeout tokens never occur in normal traffic and are not part of Σ_D. Let $T \leq 25^2$ be the number of unique event transitions stored in $\mathcal{D}_{delay}$, and let $|\mathcal{F}|$ and $\bar{N}$ be the number of normal flows used in pre-processing and their average pattern length.

5.4.1. Time complexity

Detection (Algorithm 2) The mismatch-score is updated incrementally. When a new symbol is appended to a flow's pattern, the detector performs three steps: (i) it retrieves the maximum permitted delay of the last event from $\mathcal{D}_{delay}$ and compares it with the elapsed time, which takes $O(1)$; (ii) it forms the at most n look-ahead pairs that end at the new symbol and looks each up in $\mathcal{D}_{Look\text{-}ahead}$, which takes $O(1)$ expected time per pair (hash lookup with a constant-size key), i.e., $O(n)$ in total; (iii) it updates the running mismatch counter, computes mismatch-score = mismatches / $\big(n(N-(n+1)/2)\big)$, and compares it with t, which takes $O(1)$. Appending a symbol does not change pairs already examined, so this gives the same score as re-evaluating the whole pattern, without rescanning it. Each symbol therefore costs $O(n)$, and a flow of N symbols costs $O(N \cdot n)$ in the worst case; evaluation ends earlier once the score reaches t or *Stop* is observed. Timeout tokens are counted in N and λ and cost the same as ordinary events. For F concurrent flows the aggregate cost is $O(\lambda \cdot n)$. Because flows are classified independently and both databases are read-only during evaluation, concurrent flows or attacks affect the cost only through λ. The DPI parsing cost per AMQP frame is assumed to be $O(1)$ and is common to any AMQP monitor.

Pre-processing (Algorithm 1, offline). For a normal flow with N_i symbols, extracting look-ahead pairs takes $O(N_i \cdot n)$ and building $\mathcal{D}_{delay}$ takes $O(N_i)$. The one-time cost is $O\big(\sum_i N_i\, n\big) = O(|\mathcal{F}| \cdot \bar{N} \cdot n)$, paid offline and not during detection.

5.4.2. Space complexity

Reference databases. Each entry of $\mathcal{D}_{Look\text{-}ahead}$ has the form *event[a]: event[b],k* with $k \leq n$, so

$$|\mathcal{D}_{Look\text{-}ahead}| \;\leq\; |\Sigma_D|^2 \cdot n \;=\; O(n),$$

since $|\Sigma_D|$ is a constant. This bound is independent of the number of flows and of traffic volume, and it agrees with the saturation in Fig. 9(b), where 269–1060 pairs are observed for $n = 3$–9 against bounds of 2028–6084. The set of unique event patterns (Fig. 9(a)), which keeps growing with the number of flows, is used only for analysis and is not stored by EPA. The database $\mathcal{D}_{delay}$ stores one maximum delay per transition, i.e., $O(T)$ with $T \leq 25^2$ (about 25 entries in our dataset).

Per-flow state. Both databases are read-only and shared by all flows, so the only state that grows with load is per-flow. Storing the full pattern, as in Algorithm 2, needs $O(N)$ per flow, which is unbounded for long-lived slow-rate flows that accumulate timeout tokens. It is sufficient to keep the last n symbols, the timestamp of the last event, and two counters (mismatch count and N), which needs $O(n)$ per flow.

Total. The total space is

$$O\big(|\Sigma_D|^2\, n + T + F \cdot n\big)$$

with the sliding-window buffer, or $O\big(|\Sigma_D|^2\, n + T + F \cdot \bar{N}\big)$ if full patterns are retained. The static part does not depend on traffic volume; only the $F \cdot n$ term grows with the number of concurrent flows.

6. Experiments

6.1. Dataset partitioning and evaluation protocol

To ensure a rigorous and unbiased evaluation of the proposed Event Pattern Analysis (EPA) method, the dataset is partitioned into two strictly disjoint phases: a pre-processing phase and an evaluation phase, performed at the traffic capture session level to ensure temporal separation.

In the pre-processing phase, 6106 normal AMQP flows are used exclusively to construct the look-ahead pair and maximum delay databases. These flows are derived only from legitimate traffic and are not reused during evaluation. In the evaluation phase, the dataset consists of 10,740 legitimate flows and 7321 attack flows, covering all five DoS attack variants described in Section 4.2 The legitimate flows in this phase are generated from separate capture sessions with different timestamps, message patterns, and configurations, ensuring complete independence from the pre-processing data.

Each AMQP flow is defined as a TCP connection identified by a unique five tuples, and the datasets were verified to have no overlap, ensuring no data leakage and independence. The proposed EPA method is a statistical anomaly detection approach without model training; the parameter is the mismatch-score threshold, selected via percentile based analysis (Section 6.2). Therefore, the robustness is demonstrated across multiple thresholds and window sizes.

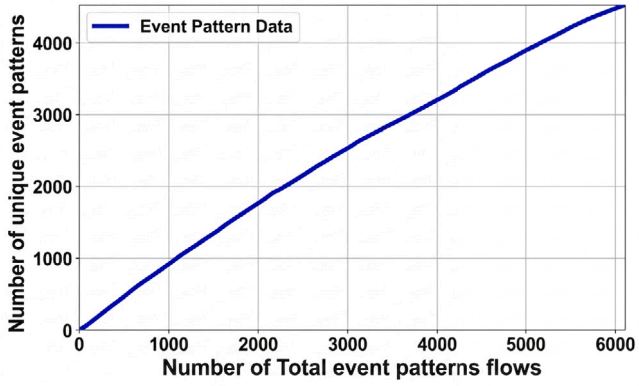
6.2. Pre-processing phase

The absence of a suitable dataset covering DoS attacks on the AMQP protocol necessitated the creation of a custom dataset for this study. Our generated dataset models both normal AMQP network traffic and a range of DoS attack variants, including Slow Incomplete Publish, Slow Partial Publish, Slow Queue Declare, Delayed Channel Close, and Basic Flooding attacks. In the pre-processing phase, the subset of normal traffic was utilized to construct the necessary reference databases for model training.

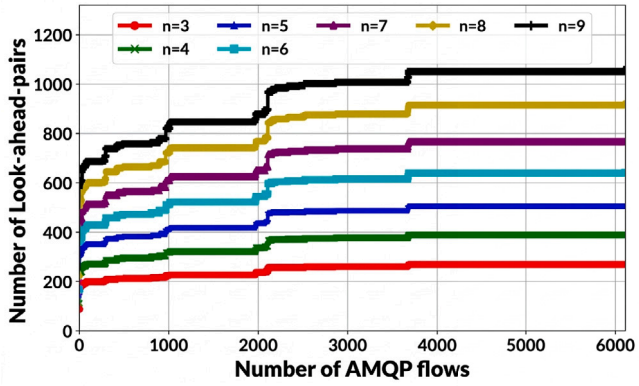
6.2.1. Obtaining the normal behavior of AMQP

Constructing a comprehensive AMQP database to encapsulate the complete normal behavior is a challenging task. To address this, we extracted 6106 normal AMQP flows from our customized dataset and generated a unique event pattern for each flow. This process resulted in approximately 4527 distinct unique event patterns. Subsequently, we derived look-ahead pairs from these unique normal event patterns using varying window sizes n: 269 pairs for window size 3, 390 for size 4, 506 for size 5, 642 for size 6, and 771 for sizes 7, 919 pairs for window size 8, and 1060 pairs for window size 9, respectively. Analysis of Figs. 9(a) and 9(b) reveals two key findings. First, as shown in Fig. 9(a), the number of unique event patterns increases continuously with the number of AMQP flows, suggesting that it is virtually infeasible to define a finite set of traces that captures the protocol's entire normal behavior using this method.

In contrast, Fig. 9(b) demonstrates that the number of new look-ahead pairs plateaus after a certain point p (approximately 4200 flows in this study). This saturation indicates that the look-ahead pairs database successfully captured nearly the complete spectrum of normal AMQP characteristics at this point. Consequently, we conclude that a database of look-ahead pairs is a more effective and efficient method for modeling the expected behavior of AMQP than a database of unique event patterns.



(a) Unique message sequences



(b) Look-ahead pairs for various window size n

Fig. 9. Collecting normal AMQP behavior.

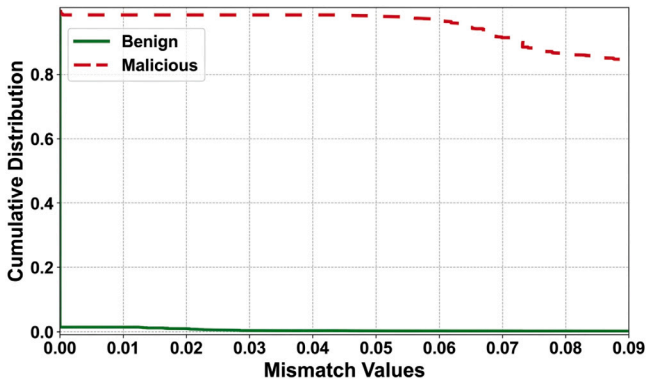


Fig. 10. Mismatch-scores of normal and attack sequences.

6.3. Evaluation phase

During the evaluation phase, we conducted experiments across four distinct time intervals while maintaining a constant window size. The primary objective was to analyze how variations in the computed mismatch-score threshold would impact the results (Tun et al., 2020). To achieve this, we employed a percentile-based threshold technique (Kumari and Tripathi, 2025; Dhiwar et al., 2025). For each time interval, the threshold was determined by considering the n percentile of mismatch-scores from normal event patterns and the $(100-n)$ percentile from attack patterns. This method allows the threshold to adapt to the statistical distribution of both normal and attack AMQP traffic.

The performance of our approach was rigorously assessed using standard metrics derived from the confusion matrix, including $Accuracy$, FPR, $Recall$, and $Precision$. As detailed in Table 10, all experiments were performed with a fixed window size of $n = 3$. The results indicate that the choice of percentile significantly influences evaluation performance. The configuration using the 92.5 percentile for normal events and the 7.5 percentile for attacks yielded the optimal balance among all evaluated metrics, and achieved the highest accuracy (99.22%), a minimal FPR (0.003), Recall (98.53%) and Precision (99.54%). This suggests that this specific threshold value effectively distinguishes between normal and attack patterns with remarkable reliability.

6.3.1. Mismatch-score of legitimate and malicious event patterns

Fig. 10 presents the Cumulative Distribution Function (CDF) of the mismatch-scores for both normal and attack traffic with a window size of $n = 3$.

Based on Fig. 10, the selection of an appropriate detection threshold is critical for balancing false positives and false negatives. The data shows that normal events consistently exhibit a very low mismatch-score, confined within the range of [0, 0.02]. Consequently, setting a threshold anywhere below 0.02 would incorrectly classify a significant number of these normal patterns as anomalous, leading to an undesirable increase in false positives FP.

To mitigate this issue, a threshold within the band of [0.03, 0.06] appears to be a more optimal choice. This range sits clearly above the upper bound of normal activity, thereby drastically reducing the number of false alarms. Furthermore, this selection is positioned before a noticeable rise in the anomalous attack scores, meaning it should effectively identify attacks without simultaneously causing a significant increase in false negatives FN. In summary, this threshold band offers a balanced trade-off, minimizing false positives while maintaining a high detection rate for true attacks.

6.3.2. Detection performance v/s window size n

To evaluate the impact of window size on detection accuracy, we conducted four separate experiments using window sizes ranging from $n = 3$ to $n = 9$. The analysis was performed on a dataset consisting of 7321 attack flows and 10,740 normal flows. A consistent threshold value was maintained across all tests to ensure a fair comparison. The results, detailed in Table 11, demonstrate that varying the window size n had no significant effect on the detection performance of the method. This indicates that the approach is robust and its accuracy remains stable across the different window configurations tested.

To further contextualize the detection performance of the proposed EPA approach, we present a quantitative comparison against widely used machine learning classifiers evaluated on the same AMQP dataset.

6.3.3. Comparative performance analysis of ML models

We conducted a comprehensive quantitative comparison between the proposed Event Pattern Analysis (EPA) framework and several widely used machine learning classifiers, including Random Forest, Logistic Regression, Gradient Boosting, and XGBoost. All methods were evaluated on the same AMQP testbed dataset using two standard performance metrics, such as detection rate and false positive rate (FPR). To ensure a fair evaluation, the experiments were performed on a unified dataset derived from the generated AMQP traffic, consisting of both legitimate and attack scenarios. The results of this comparison are summarized in Table 12.

As shown in the above table, Random Forest achieved the highest detection rate (98.96%), while the proposed EPA framework achieved a comparable detection rate of 98.53%. However, EPA significantly outperformed all machine learning models in terms of false positive rate, achieving only 0.30%, compared with 2.30% for Random Forest, 7.03% for XGBoost, 19.78% for Gradient Boosting, and 78.14% for Logistic Regression. Although several machine learning models

Table 10
Detection results.

Interval	# Flows		Mismatch-score (t)			Size of window (n)	Accuracy	FPR	Recall	Precision
	Legitimate	Malicious	Normal	Attack	Value		(%)		(%)	(%)
1	10740	7321	85	15	0.0300	3	99.20	0.003	98.53	99.50
2	10740	7321	90	10	0.0350	3	99.21	0.003	98.53	99.53
3	10740	7321	92.5	7.5	0.0400	3	99.22	0.003	98.53	99.54
4	10740	7321	95	5	0.0450	3	99.21	0.002	98.49	99.55

Table 11
Detection performance for different window sizes n.

n	Accuracy (%)	FPR	Recall (%)	Precision (%)	F1-score (%)
3	99.21	0.0029	98.49	99.55	99.018
5	99.22	0.0029	98.52	99.55	99.033
7	99.23	0.0029	98.53	99.55	99.038
9	99.24	0.0027	98.53	99.59	99.058

Table 12
Comparative analysis of detection metrics across ML models.

Method	Detection rate	False positive rate
Random Forest	0.9896	0.0230
Logistic Regression	0.9618	0.7814
Gradient Boosting	0.9752	0.1978
XGBoost	0.9544	0.0703
EPA (proposed)	**0.9853**	**0.0030**

demonstrated high detection capability, their FPRs varied considerably depending on model complexity and learning characteristics. In contrast, the proposed EPA framework maintains a favorable balance between detection performance and false alarm reduction by leveraging protocol-aware event progression and timing analysis rather than statistical model training. These results demonstrate that EPA provides competitive detection performance while substantially reducing false positives, making it well-suited for practical AMQP attack detection in controlled testbed environments.

6.3.4. Analysis of undetected attack cases

Despite the high detection performance of the proposed Event Pattern Analysis (EPA) method, a small number of attack instances remain undetected. These false negatives typically occur when attack behaviors closely mimic legitimate AMQP communication in both event patterns and timing. Since EPA detects anomalies based on deviations in look-ahead pairs and inter-event delays, attacks that preserve valid protocol flows and maintain delays within the normal bounds of D_{delay} may not generate sufficiently high mismatch-scores to cross the detection threshold.

In particular, low-intensity Slow-rate attacks (e.g., controlled slow publish) follow correct AMQP interaction steps while keeping delays within acceptable limits and avoiding timeout triggers. As a result, their mismatch-scores remain within the normal range (e.g., $[0, 0.02]$), leading to misclassification as legitimate traffic. Additionally, the variability of legitimate AMQP behavior limits the ability to capture all normal patterns, resulting in borderline cases where rare valid flows resemble attacks. This reflects a known limitation of statistical pattern based detection methods when handling stealthy adversarial behavior. A formal analysis of the evasion constraints that a mimicry attacker must satisfy is provided in Section 5.3.1, demonstrating that full evasion and effective service disruption are mutually incompatible for the attack classes considered in this work.

6.4. Sources of false positives and mitigation strategies

Section 6.3.4 examined conditions that cause false negatives. Here we consider the converse: external factors that can inflate the mismatch-score of genuinely legitimate traffic. We identify three such sources relevant to EPA's design and then outline mitigation directions.

Network congestion: Since $\mathcal{D}_{delay}$ stores the maximum inter-event delay observed during pre-processing (Section 5.2.2), a legitimate client experiencing transient congestion, retransmission, or queuing delay beyond this bound triggers a timeout token TO_m (Algorithm 2, Step 8). Each such token is, by construction, absent from $\mathcal{D}_{Look-ahead}$ (Section 5.2.1) and therefore contributes to the mismatch-score regardless of intent. Congestion that stays within the observed normal delay range does not affect classification. Using the normalization in Section 5.3.2, a single token can appear in at most $2n$ look-ahead pairs, i.e., a score contribution of at most $2/(N-(n+1)/2)$; for typical flows ($N \approx 15$–25, $n = 3$) this is roughly 0.09–0.15, above the selected threshold band. The threshold margin therefore offers little protection against delay-induced tokens, which motivates mitigations 2 and 3 below.

Device misconfiguration: Clients with altered heartbeat intervals, non-standard reconnection logic, or unusual acknowledgment behavior may produce event orderings absent from the 6106 flows used to build $\mathcal{D}_{Look-ahead}$. As noted in Section 6.2.1, the number of unique event patterns keeps growing with flow count (Fig. 9(a)) even though the look-ahead pair set plateaus (Fig. 9(b)); rare but valid configurations could therefore still yield pairs missing from the reference database.

Legitimate protocol enhancements: Broker or client-library upgrades that add AMQP capabilities (e.g., extra negotiation frames, alternate acknowledgment modes, or extended connection-tuning parameters) can shift the observed event sequence in ways not captured during pre-processing, likewise raising the mismatch-score without adversarial intent.

Scope and mitigation directions: The threshold band $[0.03, 0.06]$ was selected from the testbed's mismatch-score distributions (Section 6.3.1), where normal scores lie predominantly within $[0, 0.02]$; it gives a margin against the ordering variability present in that data, and the observed FPR is 0.2%–0.3% (Table 10). The present study did not inject congestion, misconfigured clients, or protocol-version drift, so this figure should not be extrapolated to such conditions. For deployments where they are more prevalent, we identify three design-level directions:

1. *Periodic offline retraining* of $\mathcal{D}_{Look-ahead}$ and $\mathcal{D}_{delay}$ from newly captured legitimate traffic, so that evolving client behavior and protocol features are absorbed without redesigning the detection logic (the update strategy in Section 6.6).
2. *A grace-period or hysteresis rule*, requiring the mismatch-score to stay above threshold across several consecutive pattern updates before a flow is flagged, to reduce sensitivity to short-lived congestion spikes.
3. *Session-adaptive delay bounds*, where the stored maximum in $\mathcal{D}_{delay}$ is scaled by a lightweight per-session estimate of path latency or jitter, so that transient network conditions can be distinguished from sustained, attacker-induced stalling.

These are proposed directions, not validated components of the current EPA implementation. The reported results assume a relatively stable client-broker path, consistent with the cloud-based testbed of Section 4.1. Evaluating EPA under controlled congestion and misconfiguration, and quantifying the proposed mitigations (together with the adaptive thresholding of Section 6.7), is left as future work (Section 7).

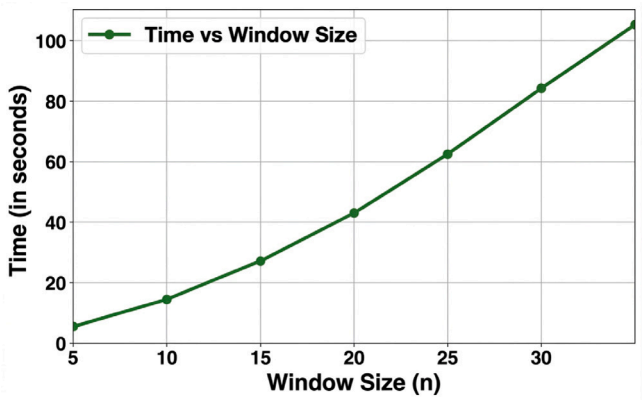


Fig. 11. Time needed to extract look-ahead pairs for various n.

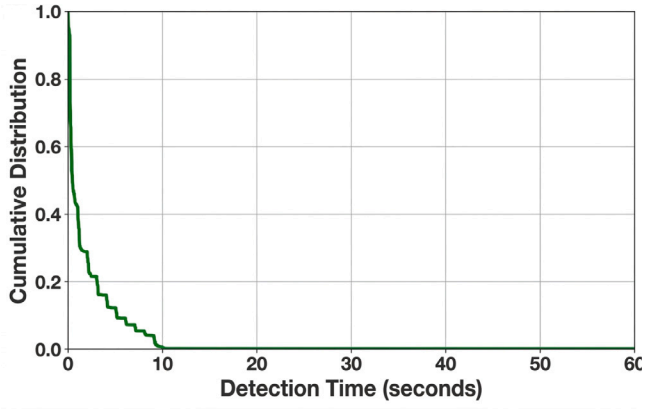


Fig. 12. CDF plot of the time required to detect malicious event patterns (for n=3).

6.5. Parameters for evaluating detection performance

Look-ahead pair extraction time:
Fig. 11 illustrates the time required to extract look-ahead pairs from the event pattern. The experimental results show a clear trend that if the window size n increases, the time required to collect these pairs also increases. This relationship highlights a critical trade-off, indicating that selecting an appropriate window size is essential for efficiently distinguishing between normal and attack event patterns.

Detection delay: An experiment was conducted to measure the detection delay, which is the time required for the proposed technique to classify an event pattern as either normal or an attack. The results, presented in Fig. 12, demonstrate that the model efficiently identifies malicious activity, achieving detection within a span of 10 s.

Processing overhead: We evaluated the computational overhead of our detection method on a system running Fedora Linux 40 (Workstation Edition), equipped with an Intel® Core™ i7-12700 processor, 32 GB of RAM, and a 256 GB SSD. To measure CPU usage, we used the `psutil` Python library (Rodola, 2009). Fig. 13 shows the CPU utilization with and without the proposed evaluation scheme over a period of 5000 s. When the evaluation process was enabled, the CPU usage remained consistently low, showing only a slight increase compared to the system running without detection. The observed difference was marginal, with usage generally staying below 5% and only a few spikes reaching around 7%–8%. These results confirm that the proposed detection approach introduces only a very small processing overhead. Therefore, it can be efficiently deployed in real-time to detect AMQP DoS attacks without significantly burdening the system resources.

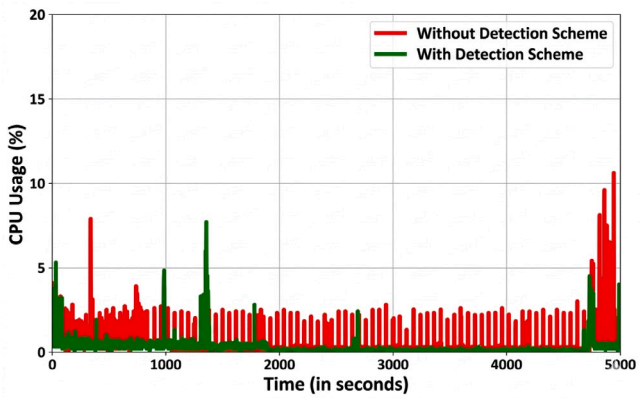


Fig. 13. CPU usage.

6.6. Feasibility on resource-constrained IoT devices

Memory footprint (analytical): The EPA approach maintains two lightweight lookup structures: a look-ahead pairs database $D_{\text{look-ahead}}$ and a maximum inter-event delay database D_{delay}. For our dataset, $D_{\text{lookahead}}$ contains at most 1060 pairs (at $n = 9$), while D_{delay} holds approximately 25 entries corresponding to unique AMQP event transitions. The total memory footprint remains well under 100 kB across all evaluated window sizes, making the approach feasible even on devices with as little as 128 MB RAM.

Scalability: To evaluate scalability, we replayed PCAP traces at 1×, 5× and 10× the original capture rate (peak of about 580 connections/min at 1×), and additionally under a *combined multi-attack workload* in which the normal traffic and all five attack variants were merged and replayed concurrently. Detection latency remained under 10 s at 1×, increased to about 13.4 s at 5×, and about 22 s at 10×. Since per-event cost is $O(n)$ and independent of the load, this increase is due to contention among the growing number of concurrent flows on the single evaluation machine (aggregate work $O(\lambda \cdot n)$), not to a change in per-flow complexity. The system remained stable without failure, and parallel per-flow processing can further reduce latency.

Memory usage (empirical): To complement the above analytical estimation, we measured the runtime memory usage using Python's `tracemalloc` module. At saturation (after processing more than 4200 training flows), the look-ahead database $D_{\text{Look-ahead}}$ with window size $n = 3$ consumed approximately 1.2 MB, while the delay database D_{delay} required less than 50 kB. The total runtime memory of the complete detection pipeline including three concurrent threads and active pattern buffers remained below 15 MB throughout all experiments. This demonstrates that the proposed approach maintains a low memory overhead in practical deployments.

Processing latency: EPA operates with a detection complexity of $\mathcal{O}(N \times n)$, making it computationally lightweight. For typical AMQP flows ($N \approx$ 15–25) and a window size of ($n = 3$), fewer than 100 comparisons are required per flow. Due to the lookup tables allow constant time matching, the method supports real time execution on resource limited devices. The CPU overhead stayed below 5% in normal operation and under 8% during peak load.

Relevance to Event Pattern based detection: Event pattern model is a well established intrusion detection strategy, with earlier works showing that normal system call or protocol event patterns form strong behavioral baselines. Recent studies extend this idea to IoT network flows. Following the same paradigm, and our model the legitimate AMQP interactions as ordered look-ahead pairs, providing lightweight

and semantically meaningful behavioral fingerprints without relying on machine learning.

Practical limitations: The EPA may require a smaller window size on devices with very limited RAM, and newly emerging attack behaviors necessitate updates to the look-ahead and delay tables. These updates are lightweight offline operations that simply regenerate the tables from new traces and deploy them to edge devices, and a periodic incremental update strategy ensures continued effectiveness against evolving attacks.

6.7. Future directions: Hybrid EPA-ML integration and cross-protocol applicability

As evaluated in this paper, EPA is a standalone statistical detector with a fixed mismatch-score threshold (Section 6.3), validated only on AMQP traffic over TCP. Two extensions are planned: integration with machine learning, and applicability to other IoT protocols. Neither has been implemented, and no results are reported for them.

Hybrid EPA-ML architecture: The comparison in Section 6.3.4 shows that ML classifiers reach a detection rate comparable to EPA but with a much higher false positive rate (2.30%–78.14% versus 0.30%). Section 6.3.4 also shows that the attacks EPA misses are low-intensity Slow-rate attacks whose mismatch-scores fall within the normal range $[0, 0.02]$, so the score alone cannot separate them from legitimate flows. We therefore plan a two-stage cascade instead of replacing EPA. Stage 1 remains unchanged. EPA computes the mismatch-score in real time at $O(N \times n)$ cost, and flows with a score $\geq t$ are flagged as attacks. Stage 2 would examine only flows that Stage 1 labels normal, using a lightweight classifier (e.g., a shallow decision tree or logistic regression) trained on the labeled flows of the proposed dataset. Candidate features are flow-level quantities that can be derived from the parsed AMQP events: connection duration, the number of channels opened but not closed, the number of `Queue.Declare` and `Queue.Bind` events per connection, the ratio of the largest observed inter-event delay to the corresponding D_{delay} entry, and the mismatch-score trajectory. Stage 2 would raise an alert only at high classifier confidence, so that EPA's low false positive rate is preserved. The planned evaluation would use the same disjoint evaluation flows as Section 6.1 and compare detection rate, FPR, detection latency, and memory against EPA alone.

Adaptive thresholding: The threshold t is currently fixed once through offline percentile analysis (Section 6.3.1). We plan to maintain a sliding-window or exponentially weighted estimate of the legitimate mismatch-score distribution and to recompute t periodically from a chosen percentile. This would allow the detector to follow gradual changes in legitimate traffic (e.g., broker reconfiguration or new client behavior) without regenerating the reference databases. The evaluation would replay traffic captured under changed client and broker configurations.

Applicability beyond AMQP: The core EPA steps, namely mapping protocol events to strings, extracting look-ahead pairs, and recording maximum inter-event delays, do not depend on AMQP in principle. Their transfer, however, requires protocol-specific event definitions:

- *AMQP* has a smaller event vocabulary (CONNECT, PUBLISH, SUBSCRIBE, PINGREQ, DISCONNECT), so the event table analogous to Table 6 would be smaller and a smaller window (e.g., $n = 2$–3) may suffice. Our earlier work on MQTT communication-sequence anomalies (Swain et al., 2025b) provides a starting point for this event mapping.
- *CoAP* is UDP-based and connectionless, so EPA's Start/Stop pattern boundary, which is anchored to the TCP handshake, does not apply directly. Flow boundaries would have to be redefined using CoAP message IDs or tokens, and the delay database would have to reflect CON/NON retransmission and timeout behavior instead of TCP-level timing.

Both extensions require new protocol-specific event mappings and new labeled datasets, which are outside the scope of this AMQP-focused study.

7. Conclusion

In our work, we addressed the critical gap in AMQP focused intrusion detection research by developing a comprehensive dataset that captures both normal traffic and five variants of DoS attacks. The existing public datasets do not capture AMQP specific application layer behavior, making it challenging to evaluate IDS performance in IoT settings. Our controlled testbed setup enabled the capture of realistic traffic involving Slow-rate, partial, and Flooding DoS attacks, forming a strong benchmark for future research. We further introduce an efficient detection model that examines AMQP communication sequences by measuring maximum delays and look-ahead event pairs across different window sizes. If the new event pattern exceeds a predefined threshold, it is identified as malicious. This approach efficiently handles temporal behaviors and avoids excessive computation, overcoming weaknesses in existing techniques. Overall, our contributions include a new AMQP DoS dataset and a robust detection strategy to advance security research in AMQP based IoT environments. The robustness analysis discusses how EPA may withstand more advanced evasion strategies, including bursty traffic patterns and gradual timing-drift attacks. This discussion is analytical only; dedicated empirical evaluation of these scenarios is left for future work. The analysis also identifies adaptive thresholding as a promising direction for improving resilience against increasingly sophisticated adversaries. Future work will evaluate EPA under realistic network conditions (e.g., variable latency and bandwidth constraints) and under simultaneous multi-attack workloads (e.g., concurrent Slow Queue Declare and Delayed Channel Close attacks against a single broker). We also plan to validate its generalizability across multiple AMQP brokers, production-scale and multi-broker deployments, and embedded, resource-constrained hardware. Concrete architectural plans for hybrid EPA-ML integration, adaptive thresholding, and cross-protocol applicability to MQTT and CoAP are detailed in Section 6.6. Finally, we will explore supplementary features, such as connection duration and resource usage, to further improve detection robustness.

CRediT authorship contribution statement

Munmun Swain: Writing-original draft, Visualization, Validation, Methodology, Conceptualization. **Sushant Kuril:** Validation, Software, Formal analysis, Data curation. **Nikhil Tripathi:** Visualization, Methodology, Conceptualization. **Kamalakanta Sethi:** Writing – review & editing, Validation, Supervision, Investigation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the Indian Institute of Technology (Indian School of Mines) Dhanbad under the Faculty Research Scheme, project MISC 0179, and by the Anusandhan National Research Foundation (ANRF), Government of India, under the Early Career Research Grant (ECRG) scheme, project number ANRF/ECRG/2025/000131/ENS.

Data availability

Data will be made available on request.

References

Al-Fayoumi, M., Al-Haija, Q.A., 2023. Capturing low-rate DDoS attack based on MQTT protocol in software defined-IoT environment. Array 19, 100316.

Alaba, F.A., Othman, M., Hashem, I.A.T., Alotaibi, F., 2017. Internet of things security: A survey. J. Netw. Comput. Appl. 88, 10–28.

Almeghlef, S.M., Al-Ghamdi, A.A.-M., Ramzan, M.S., Ragab, M., 2023. Application layer-based denial-of-service attacks detection against IoT-CoAP. Electronics 12 (12), 2563.

Alotaibi, N.S., Sayed Ahmed, H.I., Kamel, S.O.M., ElKabbany, G.F., 2024. Secure enhancement for MQTT protocol using distributed machine learning framework. Sensors 24 (5), 1638.

Alzahrani, A., Aldhyani, T.H., 2022. Artificial intelligence algorithms for detecting and classifying mqtt protocol internet of things attacks. Electronics 11 (22), 3837.

Apache ActiveMQ Project, 2004. AMQP (Apache ActiveMQ Classic) documentation. (Accessed 24 August 2025) https://activemq.apache.org/components/classic/documentation/AMQP.

Ashton, K., 1999. Kevin ashton invents the term "The Internet of Things". (Accessed 24 August 2025) https://www.historyofinformation.com/detail.php?id=3411.

Baig, Z.A., Szewczyk, P., Valli, C., Rabadia, P., Hannay, P., Chernyshev, M., Johnstone, M., Kerai, P., Ibrahim, A., Sansurooah, K., et al., 2017. Future challenges for smart cities: Cyber-security and digital forensics. Digit. Investig. 22, 3–13.

Boppana, T.K., Bagade, P., 2023. GAN-AE: An unsupervised intrusion detection system for MQTT networks. Eng. Appl. Artif. Intell. 119, 105805.

Cai, G., Cai, L., Cui, X., Lu, S., Wang, Y., Wang, X., Xue, H., 2025. Enhancing privacy and security in IoT: a CoAP protocol analysis and improvement approach: G. Cai et al. J. Supercomput. 81 (9), 1060.

Celik, Z.B., McDaniel, P., Tan, G., Godfrey, P., Zonouz, S., 2019. Program analysis of commodity IoT applications for security and privacy: Challenges and opportunities. ACM Comput. Surv. 52 (4), 1–30.

Chandola, V., Banerjee, A., Kumar, V., 2010. Anomaly detection for discrete sequences: A survey. IEEE Trans. Knowl. Data Eng. 24 (5), 823–839.

Chien, H.-Y., Wang, N.-Z., 2022. A novel MQTT 5.0-based over-the-air updating architecture facilitating stronger security. Electronics 11 (23), 3899.

Common Vulnerabilities and Exposures, 2017. CVE-2017-15699: Apache Qpid Dispatch Router — denial of service via specially crafted AMQP frame causing router crash. (Accessed 12 September 2025) https://www.cve.org/CVERecord/SearchResults?query=CVE-2017-15699.

Common Vulnerabilities and Exposures (CVE), 2015. CVE-2015-0203: Apache Qpid qpidd — denial of service via unexpected AMQP protocol sequences. (Accessed 12 September 2025) https://www.cve.org/CVERecord/SearchResults?query=CVE-2015-0203.

Common Vulnerabilities and Exposures (CVE), 2018. CVE-2018-1298: Apache Qpid Broker-J — denial of service via improper SASL authentication mechanisms (PLAIN, XOAUTH2) on certain AMQP protocols. (Accessed 12 September 2025) https://www.cve.org/CVERecord/SearchResults?query=CVE-2018-1298.

Common Vulnerabilities and Exposures (CVE), 2020. CVE-2020-27217: Eclipse Hono AMQP protocol adapter — improper validation of message size leading to denial of service. (Accessed 12 September 2025) https://www.cve.org/CVERecord/SearchResults?query=CVE-2020-27217.

Common Vulnerabilities and Exposures (CVE), 2021a. CVE-2021-22097: Spring-AMQP Remote Denial of Service via deserialization of untrusted data in toString(). (Accessed 12 September 2025) https://www.cve.org/CVERecord/SearchResults?query=CVE-2021-22097.

Common Vulnerabilities and Exposures (CVE), 2021b. CVE-2021-22116: RabbitMQ AMQP 1.0 Client Connection Endpoint — denial of service via improper input validation. (Accessed 12 September 2025) https://www.cve.org/CVERecord/SearchResults?query=CVE-2021-22116.

Common Vulnerabilities and Exposures (CVE), 2023. CVE-2023-46120: RabbitMQ Java client — memory overflow / DoS vulnerability. (Accessed 24 August 2025) https://nvd.nist.gov/vuln/detail/cve-2023-46120.

Dhiwar, C., Kumari, M.K., Tripathi, N., Joshi, P., 2025. A Statistical Anomaly Detection Approach to Detect Induced Interest Flooding Attacks in NDN. In: 17th IEEE International Conference on COMmunication Systems and NETworks. COMSNETS, pp. 1305–1310.

Dwivedi, N., Tripathi, A., 2015. Event correlation for intrusion detection systems. In: 2015 IEEE International Conference on Computational Intelligence & Communication Technology. IEEE, pp. 133–139.

Escamilla-Ambrosio, P.J., González-González, E.U., González-Duran, E., Enríquez-Ortiz, C.E., Gallegos-García, G., Lara-López, A., 2025. Coap protocol dialects for internet of thing security. In: 2025 IEEE Latin American Conference on Computational Intelligence. la-CCI, IEEE, pp. 1–6.

Fernandes, J.L., Lopes, I.C., Rodrigues, J.J., Ullah, S., 2013. Performance evaluation of restful web services and AMQP protocol. In: 2013 Fifth International Conference on Ubiquitous and Future Networks. ICUFN, IEEE, pp. 810–815.

Forrest, S., Hofmeyr, S., Somayaji, A., Longstaff, T., 1996. A sense of self for Unix processes. In: Proceedings of IEEE Symposium on Security and Privacy. pp. 120–128.

Francis, G.T., Souri, A., İnanç, N., 2024. A hybrid intrusion detection approach based on message queuing telemetry transport (MQTT) protocol in industrial internet of things. Trans. Emerg. Telecommun. Technol. 35 (9), e5030.

Hashimyar, M.E., Aiash, M., Khoshkholghi, A., Nalli, G., 2025. Signature-based security analysis and detection of IoT threats in advanced message queuing protocol. Network 5 (1).

Hekmati, A., Zhang, J., Sarkar, T., Jethwa, N., Grippo, E., Krishnamachari, B., 2024. Correlation-aware neural networks for DDoS attack detection in IoT systems. IEEE/ACM Trans. Netw. 32 (5), 3929–3944.

Ikhsanudin, M., Suryani, V., Pahlevi, R.R., 2023. DDoS attack detection on MQTT protocol using semi-supervised DBSCAN and support vector machine model. In: 2023 11th International Conference on Information and Communication Technology. ICoICT, IEEE, pp. 301–306.

Imran, Zuhairi, M.F.A., Ali, S.M., Shahid, Z., Alam, M.M., Su'ud, M.M., 2023. Improving reliability for detecting anomalies in the MQTT network by applying correlation analysis for feature selection using machine learning techniques. Appl. Sci. 13 (11), 6753.

Johansson, L., Dossot, D., 2020. RabbitMQ Essentials: Build Distributed and Scalable Applications with Message Queuing Using Rabbitmq. Packt Publishing Ltd.

Khan, M.A., Khan, M.A., Jan, S.U., Ahmad, J., Jamal, S.S., Shah, A.A., Pitropakis, N., Buchanan, W.J., 2021. A deep learning-based intrusion detection system for MQTT enabled IoT. Sensors 21 (21), 7016.

Khan, I.A., Pi, D., Kamal, S., Alsuhaibani, M., Alshammari, B.M., 2024a. Federated-boosting: a distributed and dynamic boosting-powered cyber-attack detection scheme for security and privacy of consumer IoT. IEEE Trans. Consum. Electron..

Khan, I.A., Razzak, I., Pi, D., Khan, N., Hussain, Y., Li, B., Kousar, T., 2024b. Fed-inforce-fusion: A federated reinforcement-based fusion model for security and privacy protection of IoMT networks against cyber-attacks. Inf. Fusion 101, 102002.

Khan, I.A., Razzak, I., Pi, D., Zia, U., Kamal, S., Hussain, Y., 2024c. A novel collaborative sru network with dynamic behaviour aggregation, reduced communication overhead and explainable features. IEEE J. Biomed. Health Informatics 28 (6), 3228–3235.

Khatua, M., Khurendra, V.K., Kiran, K.S., Singh, A., 2025. Machine learning-based detection and localization of communication delay attack in IoT. In: 2025 IEEE 22nd India Council International Conference. INDICON, IEEE, pp. 1–6.

Kong, L., Tan, J., Huang, J., Chen, G., Wang, S., Jin, X., Zeng, P., Khan, M., Das, S.K., 2022. Edge-computing-driven internet of things: A survey. ACM Comput. Surv. 55 (8), 1–41.

Kramer, J., 2009. Advanced message queuing protocol (AMQP). Linux J. 2009 (187), 3.

Kumar, S., Hiremath, S., Jha, P., Narang, R., 2025. Blockchain-based optimization algorithm for secure IoT communication using AMQP. In: 2025 International Conference on Ambient Intelligence in Health Care. ICAIHC, IEEE, pp. 1–6.

Kumar, H., Kumar, H., Nandanwar, H., Katarya, R., et al., 2025. Enhancing security and scalability of IoMT systems using blockchain: Addressing key challenges and limitations. In: 6th International Conference on Deep Learning, Artificial Intelligence and Robotics. ICDLAIR 2024, Atlantis Press, pp. 191–202.

Kumari, M.K., Tripathi, N., 2025. Detecting interest flooding attacks in NDN: A probability-based event-driven approach. Comput. Secur. 148, 104124.

Kuril, S., Tripathi, N., Swain, M., Sethi, K., Subudhi, P., 2026. AMQP under siege: Characterizing and measuring slow rate DoS attacks. In: 2026 18th International Conference on COMmunication Systems and NETworks. COMSNETS, IEEE, pp. 531–536.

Lee, V.C., Stankovic, J.A., Son, S.H., 2000. Intrusion detection in real-time database systems via time signatures. In: Proceedings Sixth IEEE Real-Time Technology and Applications Symposium. RTAS 2000. IEEE, pp. 124–133.

Liu, H., Dong, W., Zhu, H., Su, Z., 2024. Formal verification and security analysis of AMQP. In: 2024 IEEE 48th Annual Computers, Software, and Applications Conference. COMPSAC, IEEE, pp. 2177–2182.

Manivannan, D., 2024. Recent endeavors in machine learning-powered intrusion detection systems for the internet of things. J. Netw. Comput. Appl. 229, 103925.

MITRE Corporation, 2025. Common vulnerabilities and exposures (CVE). (Accessed 20 November 2025) https://www.CVE.org/.

Naik, N., 2017. Choice of effective messaging protocols for IoT systems: MQTT, coap, AMQP and HTTP. In: 2017 IEEE International Systems Engineering Symposium. ISSE, IEEE, pp. 1–7.

Nandanwar, H., Katarya, R., 2024a. Deep learning enabled intrusion detection system for industrial it environment. Expert Syst. Appl. 249, 123808.

Nandanwar, H., Katarya, R., 2024b. A secure and privacy-preserving IDS for IoT networks using hybrid blockchain and federated learning. In: International Conference on Next-Generation Communication and Computing. Springer, pp. 207–219.

Nandanwar, H., Katarya, R., 2025a. A hybrid blockchain-based framework for securing intrusion detection systems in internet of things. Clust. Comput. 28 (7), 471.

Nandanwar, H., Katarya, R., 2025b. Optimized intrusion detection and secure data management in IoT networks using GAO-Xgboost and ECC-integrated blockchain framework. Knowl. Inf. Syst. 1–56.

Nandanwar, H., Katarya, R., 2025c. Privacy-preserving data sharing in blockchain-enabled iot healthcare management system. Comput. J. bxaf065.

Nandanwar, H., Katarya, R., 2025d. Securing industry 5.0: An explainable deep learning model for intrusion detection in cyber-physical systems. Comput. Electr. Eng. 123, 110161.

National Institute of Standards and Technology, 2025. National vulnerability database. (Accessed 19 Novemebr 2025) https://nvd.nist.gov/.
National Institute of Standards and Technology (NIST), 2018. CVE-2018-8030: Apache Qpid Broker-J — Denial of Service via oversized AMQP 0-8/0-9/0-91 messages. (Accessed 24 August 2025) https://nvd.nist.gov/vuln/detail/CVE-2018-8030.
National Institute of Standards and Technology (NIST), 2019. CVE-2019-0200: RabbitMQ — AMQP 1.0 client does not validate "channel limit" leading to denial of service. (Accessed 12 September 2025) https://www.cve.org/CVERecord/SearchResults?query=CVE-2019-0200.
National Institute of Standards and Technology (NIST), 2021. CVE-2021-22095: RabbitMQ — denial of service via crafted AMQP frame leading to resource exhaustion. (Accessed 12 September 2025) https://www.cve.org/CVERecord/SearchResults?query=CVE-2021-22095.
National Institute of Standards and Technology (NIST), 2023a. CVE-2023-46118: RabbitMQ HTTP API — denial of service via large message publishing. (Accessed 12 September 2025) https://www.cve.org/CVERecord/SearchResults?query=CVE-2023-46118.
National Institute of Standards and Technology (NIST), 2023b. CVE-2023-46118: RabbitMQ HTTP API — denial of service via large message publishing. (Accessed 24 August 2025) https://nvd.nist.gov/vuln/detail/cve-2023-46118.
Norouzi, M., Gürkaş-Aydın, Z., 2026. A reliable cyber-attack detection architecture for cyber-physical systems in SDN-enabled internet of vehicles. Clust. Comput. 29 (2), 110.
Otoom, A.F., Abdallah, E.E., et al., 2023. Deep learning for accurate detection of brute force attacks on iot networks. Procedia Comput. Sci. 220, 291–298.
Patni, H., Darji, J., Darji, K., Gupta, R., Tanwar, S., Shahinzadeh, H., Garg, D., 2024. SmartGuardML: ML-based MQTT data analysis approach for threat prediction in smart homes. In: 2024 8th International Conference on Smart Cities, Internet of Things and Applications. SCIoT, IEEE, pp. 76–81.
Quincozes, S., Emilio, T., Kazienko, J., 2019. MQTT protocol: fundamentals, tools and future directions. IEEE Lat. Am. Trans. 17 (09), 1439–1448.
Rachmadi, S., Mandala, S., Oktaria, D., 2021. Detection of DoS attack using AdaBoost algorithm on IoT system. In: 2021 International Conference on Data Science and Its Applications. ICoDSA, IEEE, pp. 28–33.
Rodola, G., 2009. Psutil. (Accessed 25 August 2025) https://pypi.org/project/psutil/.
Roy, G.M., 2017. RabbitMQ in Depth. Manning Publisher.
Saini, K., Sharma, S., 2025. Smart road traffic monitoring: Unveiling the synergy of IoT and AI for enhanced urban mobility. ACM Comput. Surv. 57 (11), 1–45.
Sharma, S., Saini, H., 2020. Fog assisted task allocation and secure deduplication using 2FBO2 and MoWo in cluster-based industrial IoT (IIoT). Comput. Commun. 152, 187–199.
Shelby, Z., Hartke, K., Bormann, C., 2014. RFC 7252: The constrained application protocol (CoAP).
Siddharthan, H., Deepa, T., Chandhar, P., 2022. Senmqtt-set: An intelligent intrusion detection in iot-mqtt networks using ensemble multi cascade features. IEEE Access 10, 33095–33110.
Swain, M., Nishitha, C.L., Maganti, M., Sethi, K., Tripathi, N., 2025a. Leveraging Benford's law for effective intrusion detection in MQTT-IoT networks. In: 2025 17th International Conference on COMmunication Systems and NETworks. COMSNETS, IEEE, pp. 348–355.
Swain, M., Tripathi, N., Sethi, K., 2025b. Identifying communication sequence anomalies to detect DoS attacks against MQTT. Comput. Secur. 104526.
Syed, N.F., 2020. IoT-MQTT based denial of service attack modelling and detection.
Tripathi, N., 2023. Delays have dangerous ends: Slow http/2 dos attacks into the wild and their real-time detection using event sequence analysis. IEEE Trans. Dependable Secur. Comput..
Tripathi, N., Hubballi, N., 2018. Slow rate denial of service attacks against HTTP/2 and detection. Comput. Secur. 72, 255–272.
Tripathi, N., Hubballi, N., 2021. Application layer denial-of-service attacks and defense mechanisms: A survey. ACM Comput. Surv. 54 (4), 1–33.
Tripathi, N., Hubballi, N., Singh, Y., 2016. How secure are web servers? An empirical study of slow HTTP DoS attacks and detection. In: 2016 11th International Conference on Availability, Reliability and Security. ARES, IEEE, pp. 454–463.
Tripathi, N., Shaji, A.K., 2022. Defer no time, delays have dangerous ends: Slow http/2 dos attacks into the wild. In: 2022 14th International Conference on COMmunication Systems & NETworkS. COMSNETS, IEEE, pp. 194–198.
Tun, M.T., Nyaung, D.E., Phyu, M.P., 2020. Network anomaly detection using threshold-based sparse. In: Proceedings of the 11th International Conference on Advances in Information Technology. pp. 1–8.
Ullah, S., Ahmad, J., Khan, M.A., Alshehri, M.S., Boulila, W., Koubaa, A., Jan, S.U., Ch, M.M.I., 2023. TNN-IDS: Transformer neural network-based intrusion detection system for MQTT-enabled IoT networks. Comput. Netw. 237, 110072.
Vinoski, S., 2006. Advanced message queuing protocol. IEEE Internet Comput. 10 (6), 87–89.
Wiggers, S.-J., 2003. Qcon London 2025: the origin story of AMQP - advanced message queuing politics. (Accessed 11 September 2025) https://www.infoq.com/news/2025/04/origin-advanced-message-queuing/.
Zeghida, H., Boulaiche, M., Chikh, R., 2023. Securing MQTT protocol for IoT environment using IDS based on ensemble learning. Int. J. Inf. Secur. 22 (4), 1075–1086.
Zeghida, H., Boulaiche, M., Chikh, R., Bamhdi, A.M., Barros, A.L.B., Zeghida, D., Patel, A., 2025. Enhancing IoT cyber attacks intrusion detection through GAN-based data augmentation and hybrid deep learning models for MQTT network protocol cyber attacks. Clust. Comput. 28, 58.



Munmun Swain is currently pursuing her doctoral degree in Computer Science and Engineering at the Indian Institute of Information Technology, Sri City, India. She received her B.Tech. degree in Computer Science and Engineering from Biju Patnaik University of Technology, Odisha, India, in 2017, and her M.Tech. degree in Computer Science and Engineering from Veer Surendra Sai University of Technology, Burla, India, in 2020. Her research focuses on enhancing the security and reliability of IoT systems, with particular emphasis on IoT communication protocols and attack detection. She has contributed research articles to reputed journals, including Computers & Security and Cluster Computing, and has presented her research work at international conferences, including COMSNETS.



Sushant Kuril received his B.Tech in Computer Science and Engineering from IIIT Sri City, India, in 2026. His research focuses on cybersecurity, computer networks, network protocol security, and machine learning for threat detection. He specializes in analyzing application-layer protocol vulnerabilities, denial-of-service attacks, and building secure, intelligent systems. Overall, he aims to advance resilient digital infrastructures by bridging AI, networking, and security.



Nikhil Tripathi is currently working as an Assistant Professor in the Computer Science & Engineering Department at the Indian Institute of Technology (Indian School of Mines), Dhanbad, India. He earned his Ph.D. from the Indian Institute of Technology Indore, India, in 2019 and subsequently worked as a cybersecurity researcher at Fraunhofer Institute for Secure Information Technology (SIT), Germany. He primarily works on the vulnerability assessment of popular application layer protocols to explore possible DoS vulnerabilities and devise novel defense mechanisms to counter large-scale DoS attacks. He has published CVEs and research papers in journals such as IEEE Transactions on Dependable & Secure Computing, IEEE Transactions on Network Science & Engineering, ACM Computing Surveys, Computers & Security, and conferences like INFOCOM, ARES, and COMSNETS.



Kamalakant Sethi is currently working as an Assistant Professor at the National Institute of Technology Warangal, India. He completed his Ph.D. at the School of Electrical Sciences, Indian Institute of Technology Bhubaneswar, India, in 2021. Previously, he received his M.Tech. degree from the National Institute of Technology Warangal, India, in 2014, and his B Tech from the National Institute of Science and Technology, Berhampur, India, in Computer Science and Engineering in 2009. His research interests include Cryptography and Network Security. He has contributed to reputed journals, including the Journal of Information Security and Applications, Computer and Security, and Cluster Computing. He has also presented his research work at international conferences, including CANS, ANTS, ICDCN, and COMSNETS.